



Universidade Federal do Rio Grande do Norte
Centro de Tecnologia - CT
Departamento de Engenharia Elétrica - DEE
Laboratório de Automação, Controle e Instrumentação - LACI

Unidade Lógico Aritmética (ULA)

ELE3515 - Grupo 01 - Problema 01 - Implementação

Líder:	Karen Helloisa Araújo Silva
Redator:	Eduardo Lira da Silva Filho
Debatedor:	Rinaldo Tavares da Silva Filho

Natal, 28 de agosto de 2025

Resumo

Este trabalho apresenta o projeto e a implementação de uma Unidade Lógica Aritmética (ULA) no contexto da disciplina de Sistemas Digitais. O objetivo foi desenvolver um circuito capaz de executar um conjunto de operações aritméticas e lógicas, com duas entradas de 8 bits, uma entrada de controle de 4 bits e saídas compostas por um barramento de 8 bits e dois sinais de status (Carry e Zero). O método adotado envolveu o estudo de blocos funcionais fundamentais, como somadores, multiplexadores, deslocadores, comparadores e conversores, seguido da construção das funções definidas pela especificação e da integração dos módulos em um sistema completo. A implementação considerou a otimização por mapas de Karnaugh e a utilização do algoritmo Double-Dabble para conversão binário-BCD. Os resultados obtidos demonstraram a correta execução das operações propostas, confirmando a viabilidade do projeto e o aprendizado prático dos conceitos de circuitos digitais.

Palavras-chave: Unidade Lógica Aritmética; Sistemas Digitais; Circuitos Combinacionais; VHDL.

Sumário

1	Introdução	5
2	Fundamentação Teórica	7
2.1	Portas Lógicas	7
2.1.1	NOT	7
2.1.2	OR	7
2.1.3	AND	8
2.1.4	NAND	8
2.1.5	NOR	8
2.1.6	XOR	8
2.1.7	XNOR	9
2.2	Somadores	9
2.3	Multiplexadores	10
2.4	Deslocadores	12
2.4.1	Deslocador simples	12
2.4.2	Deslocador barrel	13
2.5	Comparadores	13
2.6	Algoritmo Double-Dabble	14
2.7	Otimização por mapa de Karnaugh	15
3	Desenvolvimento da Solução	16
3.1	Projeto	16
3.1.1	Função 1 - ADD A,B	16
3.1.2	Função 2 - SUB A,B	17
3.1.3	Função 3 - MUL A,B	17
3.1.4	Função 4 - INC A	18
3.1.5	Função 5 - DEC A	19
3.1.6	Função 8 - SHL A,B	20
3.1.7	Função 9 - SHR A,B	20
3.1.8	Função 10 - SWP A	21
3.1.9	Função 11 - SWA A	21
3.1.10	Função 12 - AND A,B	22
3.1.11	Função 13 - OR A, B	22
3.1.12	Função 14 - XOR A, B	23
3.1.13	Função 15 - NOT A	24
3.1.14	Conversor BIN-BCD	24
3.1.15	Conversor BDC para 7 segmentos	25
3.1.16	Multiplexador 16x9	27

3.1.17	Comparador do Zero	29
4	Implementação	30
4.0.1	Função 1 - ADD A,B	30
4.0.2	Função 2 - SUB A,B	31
4.0.3	Função 3 - MUL A,B	32
4.0.4	Função 4 - INC A	32
4.0.5	Função 5 - DEC A	33
4.0.6	Função 8 - SHL A,B	33
4.0.7	Função 9 - SHR A,B	34
4.0.8	Função 10 - SWP A	34
4.0.9	Função 11 - SWA A	34
4.0.10	Função 12 - AND A,B	35
4.0.11	Função 13 - OR A, B	35
4.0.12	Função 14 - XOR A, B	35
4.0.13	Função 15 - NOT A	35
4.0.14	Conversor BDC para 7 segmentos	35
4.0.15	Double Dabble	36
4.0.16	Multiplexador 16x9	36
4.0.17	Correções do projeto	37
4.0.18	Montagem do Sistema	38
5	Resultados	40
5.1	Bloco de Funções	40
5.2	Conversor BIN-7-SEG	41
5.3	Gravação e testes	41
5.3.1	Figuras	43
6	Conclusão	45
A	Relato semanal - Semana 1	47
A.1	Equipe	47
A.2	Defina o problema	47
A.3	Pontos-chaves	47
A.4	Questões de pesquisa	47
A.5	Planejamento da pesquisa	47
B	Relato semanal - Semana 2	48
B.1	Equipe	48
C	Anexos	49

1 Introdução

No contexto da disciplina de Sistemas Digitais, o desenvolvimento de circuitos lógicos que implementem funções aritméticas e lógicas é fundamental para compreender a base de funcionamento dos processadores modernos.

A Unidade Lógica e Aritmética (ULA) é um componente de bloco operacional que constitui um dos blocos mais importantes de um processador, capaz de executar diversas operações aritméticas e lógicas com duas entradas de dados. Alguns exemplos de operações lógicas incluem AND, OR, XOR, etc.; algumas operações aritméticas incluem adição, multiplicação e divisão (Vahid 2008).

O problema proposto consistiu no projeto de implementação de uma ULA com duas entradas de 8 bits e uma entrada de controle de 4 bits, capaz de realizar um conjunto de instruções definidas em tabela, e apresentar os resultados em uma saída de 8 bits e em duas saídas indicadores de status (Carry e Zero). Com relação a saída de 8 bits, o circuito deverá apresentar o valor da saída da ULA através dos displays de 7 segmentos, mas antes, a saída binária deve ser convertida na representação BCD.

O fluxograma do funcionamento da ULA apresentada é ilustrada na Fig.1.1.

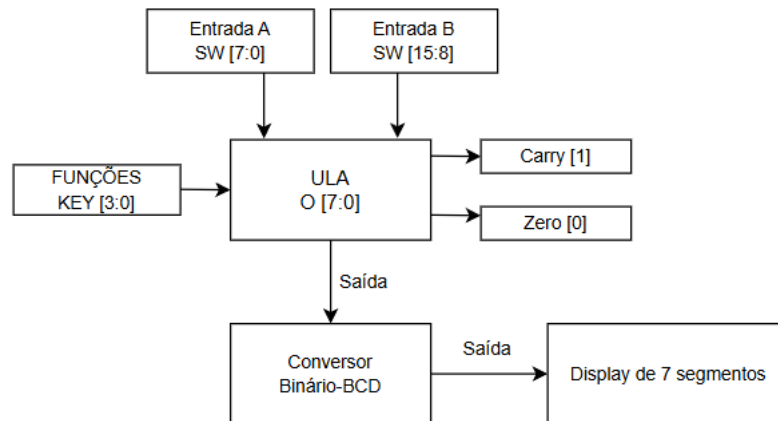


Figura 1.1: Fluxograma da ULA

O projeto da ULA foi dividido em três componentes principais, além das funções, para facilitar a implementação: (i) o multiplexador 16x9, (ii) o conversor BIN-BCD e (iii) o conversor BCD para sete segmentos. As entradas externas são compostas por $A \in \{0, 1\}^8$, $B \in \{0, 1\}^8$ e os bits de seleção $KEY[3:0] \in \{0, 1\}^4$, totalizando 20 bits.

Os 8 bits de A e B alimentam, em paralelo, as 13 funções da ULA. Cada função produz um vetor de 9 bits $S_i[7:0]$ (com $i \in \{0, \dots, 12\}$), além dos sinais de *Carry*

quando aplicável. Esses 13 vetores são conectados ao multiplexador 16x9, de modo que a combinação KEY[3:0] seleciona exatamente um dos vetores de saída das funções.

A implementação do multiplexador 16x9 é realizada por **nove** multiplexadores 16x1 em paralelo. Para o bit menos significativo, por exemplo, o primeiro multiplexador recebe as entradas $\{S_0[0], S_1[0], \dots, S_{12}[0], 0, 0, 0\}$, onde os três canais restantes são mantidos em nível lógico nulo. O mesmo arranjo se repete para os demais bits ($k = 0, \dots, 7$), garantindo que a saída do multiplexador coincide exatamente com a função selecionada por KEY[3:0]. O Diagrama implementado pode ser observado na Fig. 1.2.

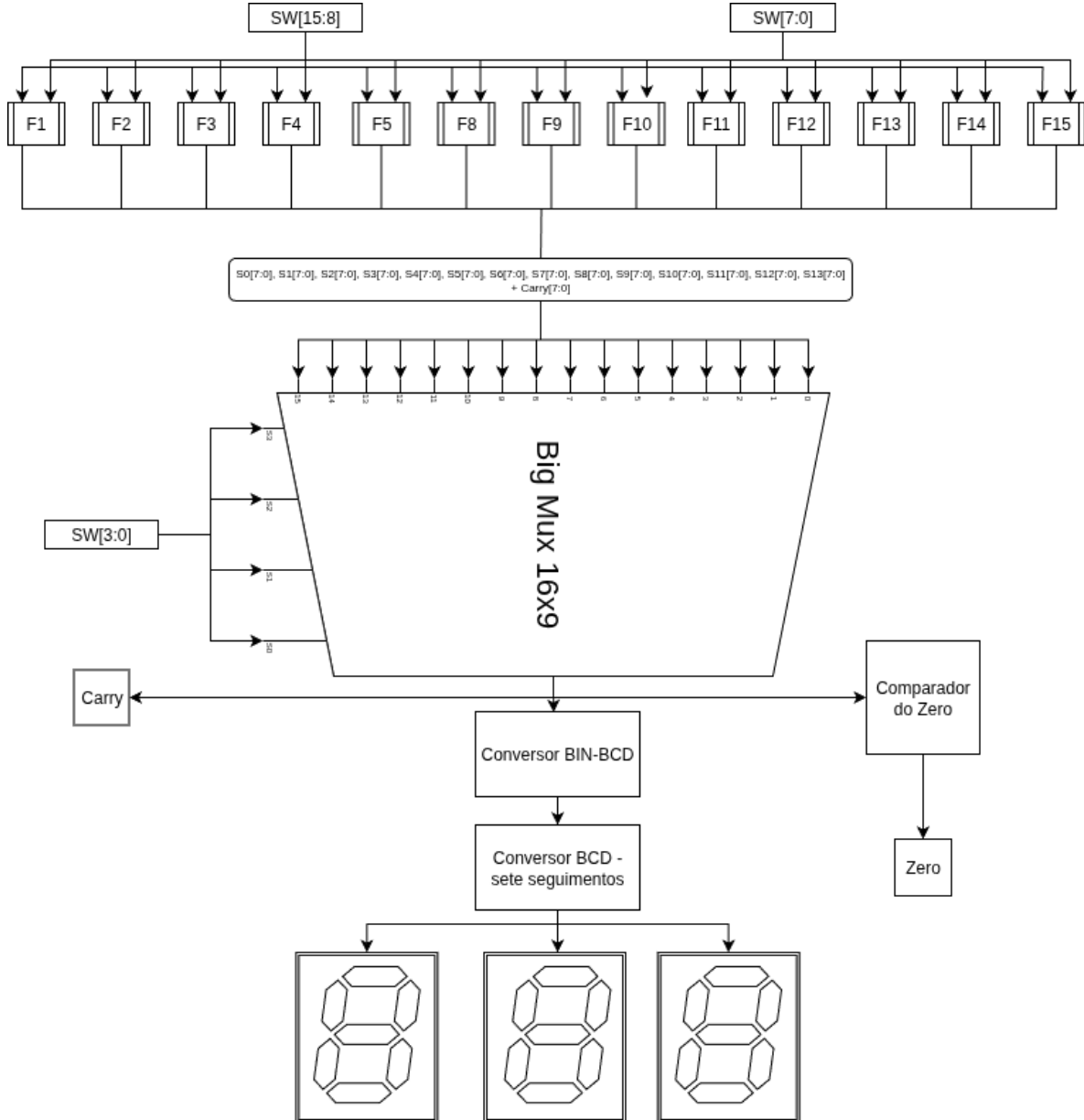


Figura 1.2: Diagrama da ULA

2 Fundamentação Teórica

2.1 Portas Lógicas

As portas lógicas constituem elementos fundamentais na construção de circuitos digitais. Elas recebem sinais de entrada binários, representados pelos níveis lógicos 0 (falso ou baixo) e 1 (verdadeiro ou alto), e produzem uma saída definida por uma função lógica específica, conforme os princípios da álgebra booleana ([Vahid 2008](#)).

A seguir serão mostrada as portas lógicas utilizadas em circuitos digitais e suas respectivas tabelaS-verdade.

2.1.1 NOT

A porta NOT produz 1 quando seu operando é 0. De igual modo, ela retorna 0 quando seu operando é 1. Em virtude deste comportamento, é comumente chamado de porta lógica inversora. Sua tabela-verdade e representação gráfica estão ilustradas pela Tabela 2.1 e Fig. 2.1.

A	S
0	1
1	0

Tabela 2.1: Porta NOT

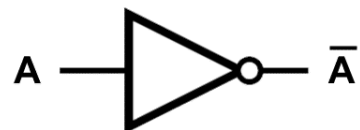


Figura 2.1: Símbolo da Porta NOT

2.1.2 OR

A porta OR produz 1 quando um ou ambos os operando são 1. Sua tabela-verdade e representação gráfica estão ilustradas pela Tabela 2.2 e Fig. 2.2.

A	B	S
0	0	0
0	1	1
1	0	1
1	1	1

Tabela 2.2: Porta OR

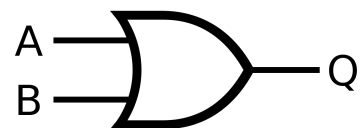


Figura 2.2: Símbolo da Porta OR

2.1.3 AND

A porta AND produz 1 quando ambos os operandos são 1. Sua tabela-verdade e representação gráfica estão ilustradas pela Tabela 2.3 e Fig. 2.3.

A	B	S
0	0	0
0	1	0
1	0	0
1	1	1

Tabela 2.3: Porta AND



Figura 2.3: Símbolo da porta AND

2.1.4 NAND

A porta NAND tem saída oposta de uma porta AND, produzindo 0 quando todas as entradas são 1, e 1 se qualquer uma das entradas for 0. Um porta NAND tem o mesmo comportamento de uma porta AND seguida de uma NOT. Sua tabela-verdade e representação gráfica estão ilustradas pela Tabela 2.4 e Fig. 2.4.

A	B	S
0	0	1
0	1	1
1	0	1
1	1	0

Tabela 2.4: Porta NAND

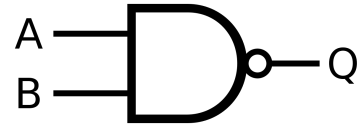


Figura 2.4: Símbolo da Porta NAND

2.1.5 NOR

Uma porta NOR tem a saída oposta de uma OR, produzindo um 0 quando no mínimo uma das entradas é 1, e 1 se todas as entradas forem 0. Uma porta NOR tem o mesmo comportamento de uma porta OR seguida de uma NOT. Sua tabela-verdade e representação gráfica estão ilustradas pela Tabela 2.5 e Fig. 2.5.

A	B	S
0	0	1
0	1	0
1	0	0
1	1	0

Tabela 2.5: Porta NOR

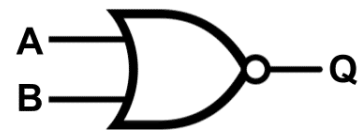


Figura 2.5: Símbolo da Porta NOR

2.1.6 XOR

Uma porta XOR produzirá uma saída 1 se exatamente uma das entradas for 1. Sua tabela-verdade e representação gráfica estão ilustradas pela Tabela 2.6 e Fig. 2.6.

A	B	S
0	0	0
0	1	1
1	0	1
1	1	0

Tabela 2.6: Porta XOR

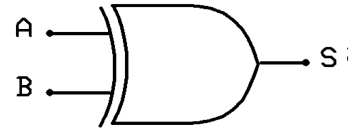


Figura 2.6: Símbolo da Porta XOR

2.1.7 XNOR

A porta XNOR é oposta de uma XOR. Para produzir 1, todos os operandos devem ser obrigatoriamente iguais. Sua tabela-verdade e representação gráfica estão ilustradas pela Tabela 2.7 e Fig. 2.7.

A	B	S
0	0	1
0	1	0
1	0	0
1	1	1

Tabela 2.7: Porta XNOR

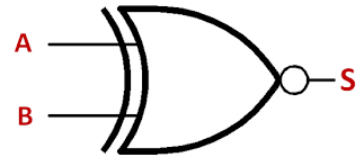


Figura 2.7: Símbolo da Porta XNOR

2.2 Somadores

Um somador é um componente de bloco operacional que adiciona dois números binários A e B de N bits gerando uma soma de N bits e um bit de transporte (bit de carry). Neste caso, A e B devem ter necessariamente o mesmo número N de bits. (Vahid 2008)

A abordagem comumente utilizada para projetar um circuito lógico combinacional capaz de somar dois números binários consiste na criação de um circuito que imita a forma pela qual nós somamos à mão dois números, utilizando uma coluna de cada vez. A Fig. 2.8 ilustra este processo manual de soma, no qual para somar dois números A e B de 4 bits, no qual $A = 1111$ (15 na base 10) e $B = 0110$ (6 na base 10).

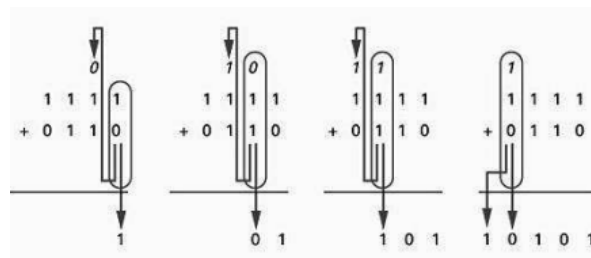


Figura 2.8: Adição de dois números binários coluna a coluna

Fonte: VAHID, 2008, p. 183.

Nota-se que ao longo da soma dos 3 números menos significativos é gerado bits de carry in C_i , e na soma do primeiro bit mais significativo pode ser gerado o bit de carry

out Co .

Para projetar este circuito somador de 4 bits, é utilizada uma combinação de portas AND, OR e XOR. Cujas tabelas verdade é descrita na Tabela 2.8.

Tabela 2.8: Tabela-verdade de um somador de 4 bits

A	B	Ci	S	Co
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Através da simplificação utilizando o Mapa de Karnaugh, encontramos a expressão para Co utilizando 3 portas AND ligadas a 1 porta OR:

$$Co = ACi + BCi + AB$$

Já a expressão para a saída S é dada por duas portas XOR conectadas:

$$S = A \oplus B \oplus Ci$$

Deste modo, para construir um somador de 4 bits, basta conectar o circuito resultante destas expressões lógicas em cascata. De modo semelhante, para construir um somador de 8 bits, é necessário que sejam utilizados 2 somadores de 4 bits em cascata como ilustra a Fig. 2.9.

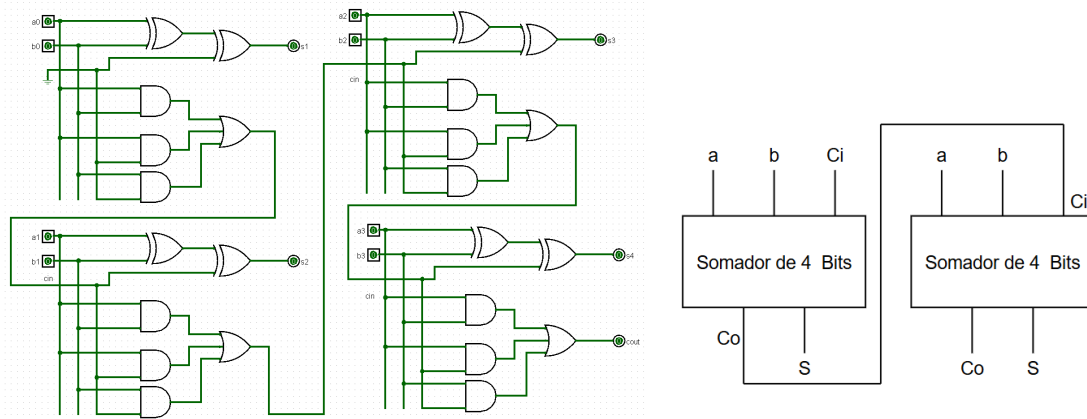


Figura 2.9: Somador de 4 Bits

2.3 Multiplexadores

Multiplexadores são componentes que recebem diversas entradas e seleciona uma delas para transferir a saída. Também são chamados de seletores porque selecionam

umas das entradas para ser passada à saída. Este controle é feito através de uma entrada de seleção.

Um multiplexador de 2 entradas, conhecido como multiplexador 2x1, tem duas entradas de dados $i1$ e $i0$, uma entrada de seleção $s0$ e uma saída de dados d . Se $s0 = 0$, o valor de $i0$ passará para a saída. Se $s0 = 1$, o valor que passará será o de $i1$. A tabela verdade que representa este componente está ilustrada abaixo.

Tabela 2.9: Tabela-verdade de um MUX 2x1

$s0$	$i1$	$i0$	d
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Cuja a expressão para a saída d é dada pela seguinte expressão lógica:

$$d = s0'i0 + s0i1$$

A representação deste bloco está ilustrada na Fig. 2.10.

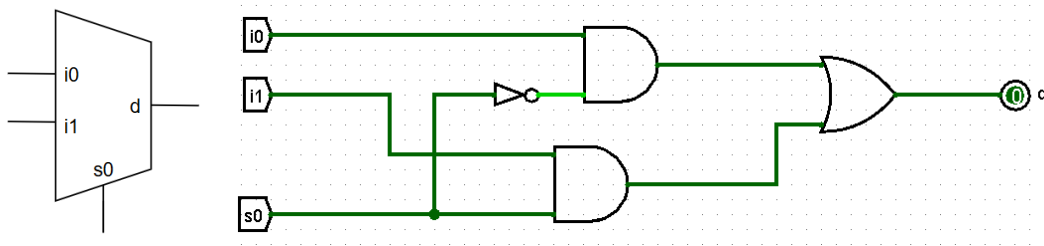


Figura 2.10: MUX 2x1

Para construir um multiplexador de 16 bits de entrada e 1 bit de saída, é necessário gerar uma rede de MUX 2x1. A representação deste bloco está ilustrada na Fig. 2.11.

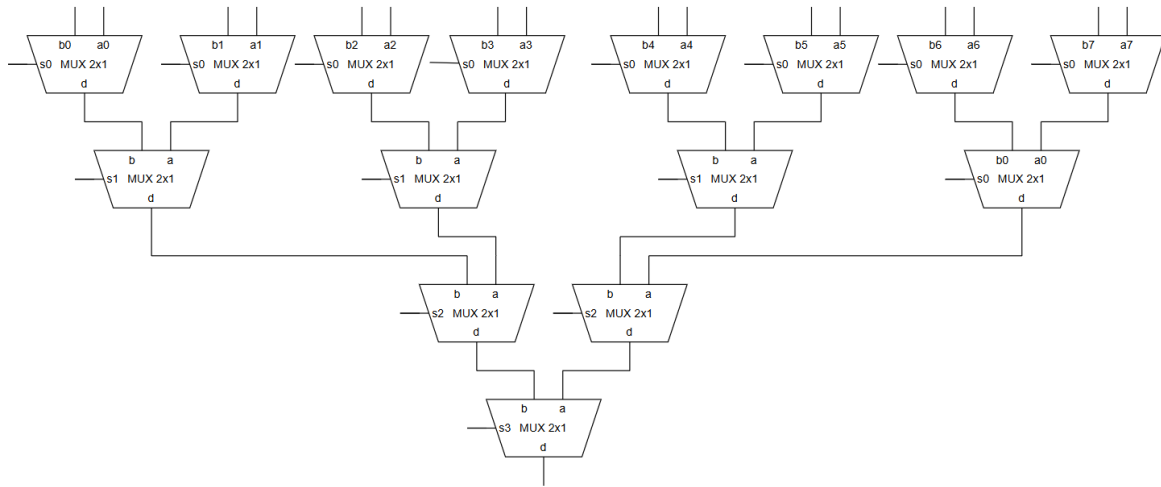


Figura 2.11: MUX 16x1

De igual modo, para desenvolver um multiplexador de 16 bits de entrada e 8 bits de saída, basta conectar em paralelo oito componentes MUX 16x1 para selecionar todos os bits da saída.

2.4 Deslocadores

O deslocamento de bits é uma operação fundamental na manipulação de dados digitais, permitindo alterar a posição dos bits em um número. Essa operação é útil para várias aplicações, como inverter a ordem dos bits, transmitir dados serialmente ou realizar operações aritméticas de forma eficiente. Na base binária, deslocar os bits à esquerda equivale a multiplicar o número por 2 para cada posição deslocada, enquanto deslocar à direita corresponde a dividi-lo por 2. (Vahid 2008).

Além do uso aritmético, o deslocamento de bits pode ser implementado tanto por registradores de deslocamento quanto por componentes combinacionais específicos, dependendo da necessidade de deslocar números diferentes em várias posições e em ambos os sentidos. Essa flexibilidade facilita operações rápidas de multiplicação, divisão (Vahid 2008).

2.4.1 Deslocador simples

O deslocador mais simples realiza um deslocamento de apenas uma posição em um dado sentido. Suponhamos que desejemos um deslocador que faça um deslocamento de uma posição à esquerda. O projeto desse deslocador simples é direto, consistindo apenas de fios interconectados, como mostrado na Fig. 2.12 (Vahid 2008).

Para operar, cada bit de entrada é conectado à posição de saída imediatamente à sua esquerda, ou a direita dependendo do tipo de deslocador, enquanto a posição menos significativa é preenchida com zero ou com um valor específico, dependendo da aplicação.

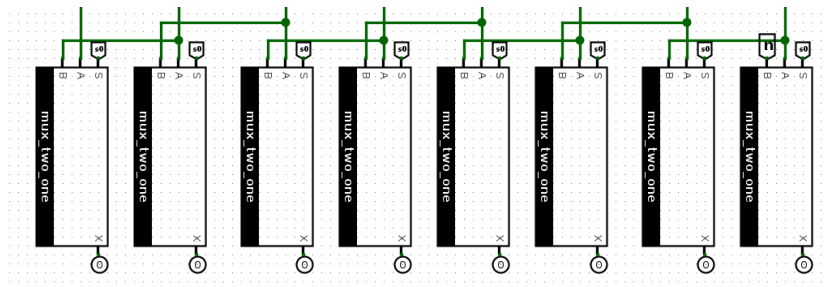


Figura 2.12: Deslocador simples de oito bits

2.4.2 Deslocador barrel

Um deslocador *barrel* de N bits é um deslocador de uso geral que pode realizar deslocamentos ou rotações de qualquer número de posições. Um projeto de um deslocador *barrel* de oito bits pode ser obtido com três deslocadores simples dispostos em cascata, como mostrado na Fig. 2.13. O primeiro pode realizar deslocamentos de quatro (ou nenhuma) posição à esquerda, o segundo de duas (ou nenhuma) posições à esquerda, e o terceiro de uma (ou nenhuma) posição à esquerda (Vahid 2008).

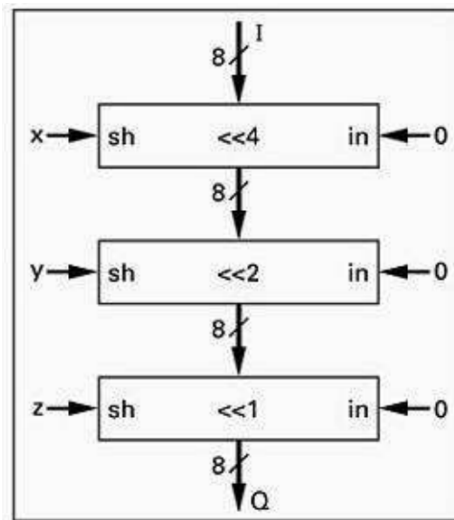


Figura 2.13: Deslocador barrel de oito bits

2.5 Comparadores

Frequentemente é necessário comparar dois números binários para verificar se são iguais ou identificar qual deles é o maior. Para isso, utilizamos o **comparador de magnitude**, um circuito combinacional que recebe como entrada dois números binários A e B e gera como saída três condições lógicas: $A > B$, $A = B$ ou $A < B$.

Um deslocador binário pode ser implementado com a logica presente na Fig. 2.14.

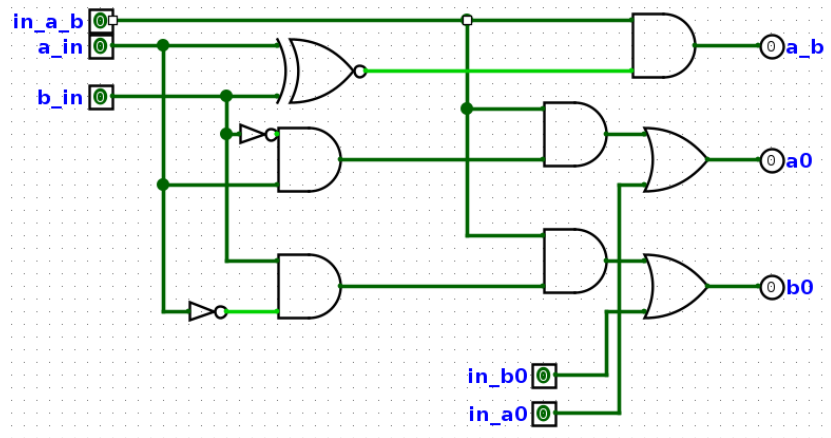


Figura 2.14: Comparador de 1 bit

Para um comparador de n basta empilhar esse circuito binário n vezes. A comparação é realizada de forma hierárquica, iniciando-se pelos bits mais significativos e avançando para os menos significativos. Sempre que os bits de mesma ordem forem iguais, a decisão é adiada para o próximo par de bits. Assim que for encontrada uma diferença, a comparação se encerra, determinando qual número é maior.

2.6 Algoritmo Double-Dabble

O algoritmo Double-Dabble é um método utilizado para converter números binários em BCD (Binary Coded Decimal) por meio de deslocamentos e ajustes simples. O circuito consiste em realizar sucessivos deslocamentos à esquerda do valor binário e, sempre que um dígito BCD parcial atingir valor superior a 4, soma-se 3 antes do próximo deslocamento. O processo é repetido até que todos os bits do número binário tenham sido utilizados, resultando nos dígitos decimais codificados em BCD. (Falconer 2004)

A tabela 2.10 descreve de forma sequencial o algoritmo Double-Dabble para converter o número 255 em binário para o seu equivalente em BCD.

Tabela 2.10: Execução do algoritmo Double-Dabble para conversão de 255 em BCD

HUNDREDS	TENS	UNITS	BINARY	Ação
0000	0000	0000	11111111	Start
0000	0000	0001	11111110	Shift 1
0000	0000	0011	11111100	Shift 2
0000	0000	0111	11111000	Shift 3
0000	0000	1010	11110000	ADD-3 to UNITS
0000	0001	0101	11110000	Shift 4
0000	0001	1000	11110000	ADD-3 to UNITS
0000	0011	0001	11100000	Shift 5
0000	0110	0011	11000000	Shift 6
0000	1001	0011	11000000	ADD-3 to TENS
0001	0010	0111	10000000	Shift 7
0001	0010	1010	10000000	ADD-3 to UNITS
0010	0101	0101	00000000	Shift 8

2.7 Otimização por mapa de Karnaugh

Os mapas de Karnaugh, ou simplesmente mapas K, constituem um método visual cujo objetivo é auxiliar na minimização algébrica de equações booleanas que envolvem poucas variáveis (tipicamente de duas a quatro) ([Vahid 2008](#)).

Esse método organiza os termos da função em uma grade bidimensional, Fig. 2.15, permitindo a identificação de padrões e agrupamentos que correspondem a simplificações possíveis.

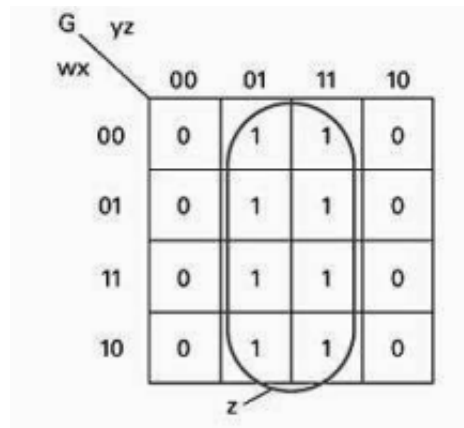


Figura 2.15: Mapa de Karnaugh

Dessa forma, o mapa de Karnaugh reduz o esforço de manipulação algébrica, facilitando a obtenção de expressões mais compactas, que se traduzem em implementações lógicas com menor custo em termos de portas e interconexões.

3 Desenvolvimento da Solução

3.1 Projeto

3.1.1 Função 1 - ADD A,B

A primeira função realiza a operação de **soma** entre as duas entradas A e B . A saída é obtida através de:

$$O = A + B$$

Essa operação utiliza um somador aritmético. Para isso, foram empregados dois somadores de 4 bits em cascata. A operação é realizada corretamente porque o bit de carry de saída do primeiro somador está conectado à entrada de carry do segundo somador. A representação gráfica do circuito é observada na Fig. 3.1.

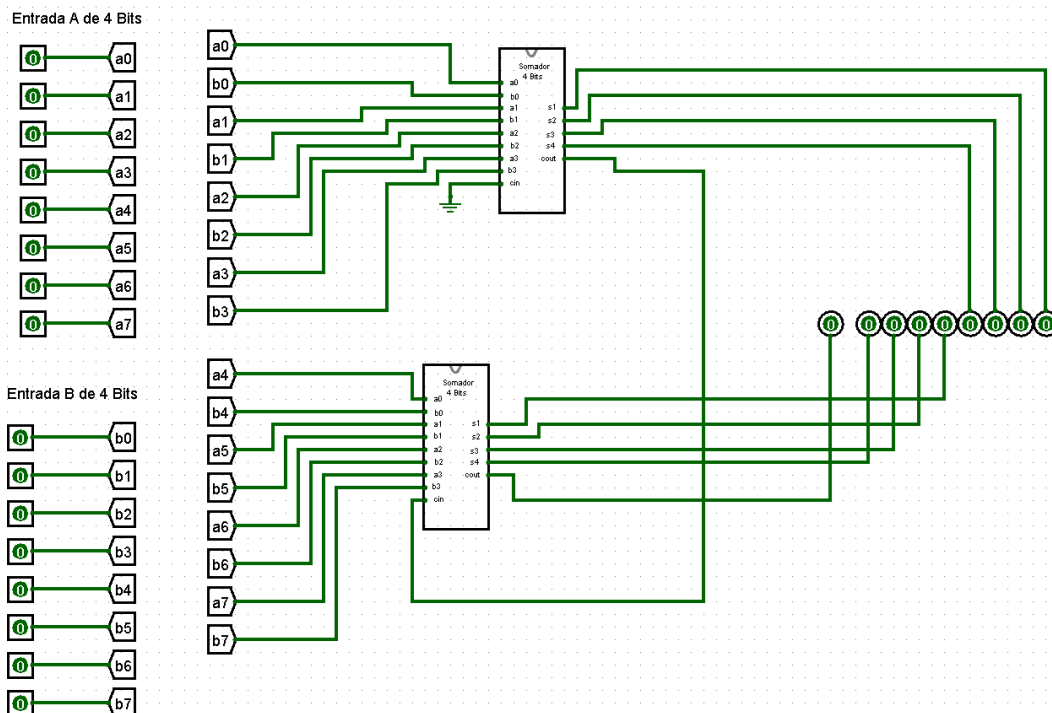


Figura 3.1: Função 1 – ADD A,B

Como ambas as entradas são formadas por 8 bits, a saída desta operação também deve gerar 8 bits. No caso da operação gerar o bit de carry out através da soma dos dois bits mais significativos dos operandos, é reservada uma saída destinada a ele.

3.1.2 Função 2 - SUB A,B

A segunda função realiza a operação de **subtração** entre as duas entradas A e B . A saída é obtida através de:

$$O = A - B$$

A operação de subtração apresenta implementação relativamente simples. Para sua realização, aplica-se o complemento de dois a um dos operandos, o que consiste em inverter todos os bits por meio de portas NOT e, em seguida, adicionar 1 ao resultado obtido.

Posteriormente, realiza-se a soma desse valor com o outro operando, utilizando o somador previamente descrito neste relatório. No circuito desenvolvido, foram empregadas oito portas NOT para a inversão do primeiro operando, enquanto a adição do bit “1” necessária para o complemento de dois foi implementada por meio da aplicação de nível lógico alto na entrada de carry in do somador.

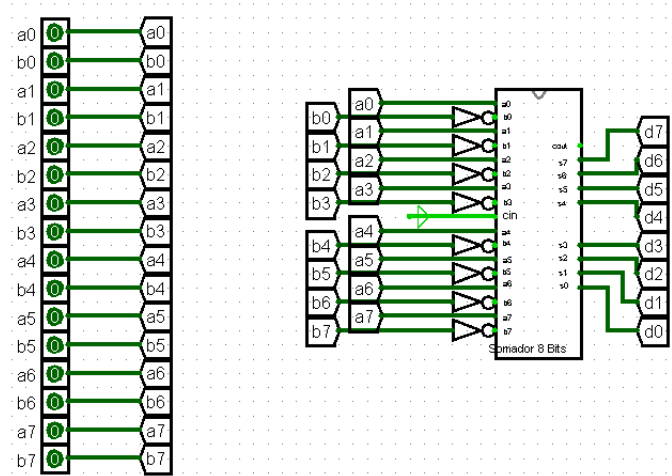


Figura 3.2: Função 2 - SUB A,B

3.1.3 Função 3 - MUL A,B

A terceira função realiza a operação de **multiplicação** entre o operando A e o operando B . Ou seja a saída O deve ser o valor da multiplicação entre as duas entradas de 8 bits, representada por:

$$O = (A * B)[7 : 0]$$

A lógica do circuito segue o mesmo padrão descrito por (Vahid 2008), a diferença sendo que nesse é um multiplicador de 8 bits, sendo assim, é necessário o uso de somadores de 16 bits.

Segundo descrito por (Vahid 2008) as entradas geram uma matriz de 8x8 bits, sendo cada bit dessa matriz o resultado da porta lógica AND, como segue descrito na Tab. 3.1. Após essa operação, é realizada a soma de cada linha da tabela deslocada para esquerda x vezes, sendo o valor de x igual ao índice da linha, como representado na

Tab. 3.2. Como é esperado uma saída de 8 bits é aproveitado apenas os 8 menos significativos.

Tabela 3.1: Matrix binária AND (8x8)

AND	1	0	0	0	1	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
1	1	0	0	0	1	0	0	0
1	1	0	0	0	1	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
1	1	0	0	0	1	0	0	0
1	1	0	0	0	1	0	0	0

Tabela 3.2: Representação da multiplicação com 16 bits

Linha	Original (bin)	Deslocado (bin)
L0	00000000	0000000000000000
L1	00000000	0000000000000000
L2	10001000	0000001000100000
L3	10001000	0000010001000000
L4	00000000	0000000000000000
L5	00000000	0000000000000000
L6	10001000	0010001000000000
L7	10001000	0100010000000000
Resultado	—	0110110001100000

3.1.4 Função 4 - INC A

Esta função realiza a operação de **incremento** da entrada A . A saída 0 é obtida somando 1 ao valor binário representado por este número.

$$O = A + 1$$

Esta função reutiliza o somador aritmético e operando adicionando 1 ao valor presente na entrada A . A representação gráfica deste circuito pode ser observada na Fig. 3.3.

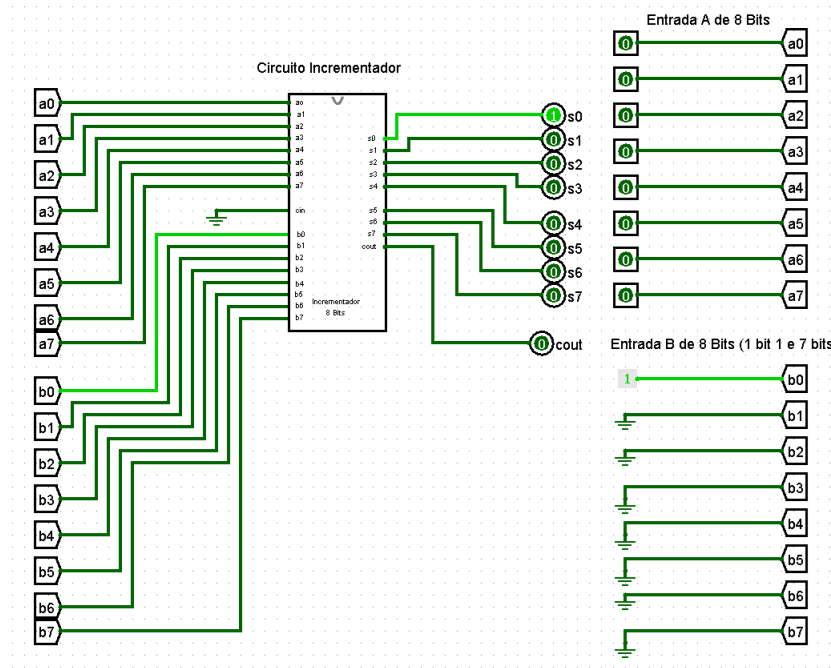


Figura 3.3: Função 4 – INC A

3.1.5 Função 5 - DEC A

A função de número 5 realiza a operação de **decremento** no operando A . Isso significa que a saída O é obtida subtraindo 1 do valor de A :

$$O = A - 1$$

Essa operação reutiliza o somador aritmético e opera adicionando o numero -1 ao valor presente na entrada A . A representação gráfica do circuito pode ser observada na Fig. 3.1.

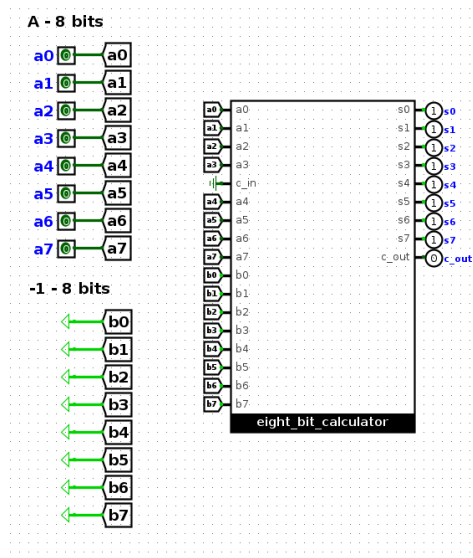


Figura 3.4: Função 5 – DEC A.

3.1.6 Função 8 - SHL A,B

A função de número 8, $O = A \ll B[2 : 0]$, $C = \text{Msb}$, é um deslocador de 8 bits para a esquerda, em que a entrada consiste nos bits de **A** e o número de posições a serem deslocadas é determinado pelos três bits menos significativos de **B**.

A sua implementação foi realizada através de um deslocador *barrel* de 8 bits, composto por quatro deslocadores simples dispostos em cascata: o primeiro desloca 8 posições à esquerda, o segundo desloca 4 posições, o terceiro desloca 2 posições e o último desloca 1 posição. O bit de carry (C) corresponde ao bit mais significativo resultante do deslocamento.

A circuito que implementa as função 8 esta representado na Fig. 3.5.

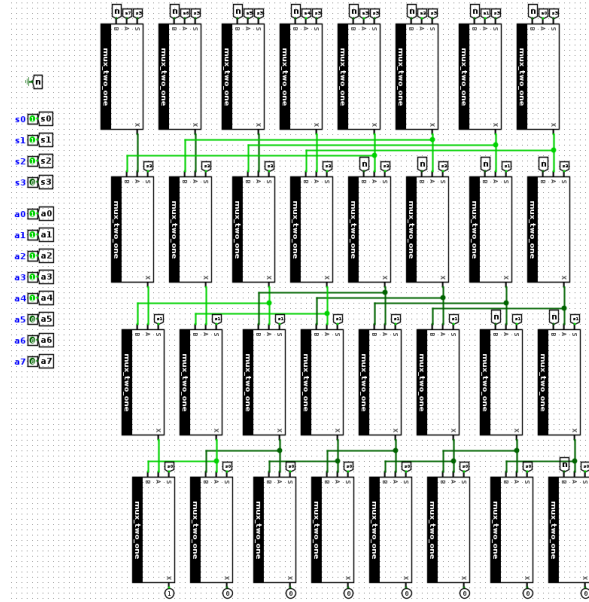


Figura 3.5: Função 8 – SHL A, B

3.1.7 Função 9 - SHR A,B

A função numero 9 $O = A \gg B[2 : 0]$ é um deslocador de 8 bits para a direita, em que a entrada consiste nos bits de **A** e o número de posições a serem deslocadas é determinado pelos três bits menos significativos de **B**. Essa é similar a função 8, no entanto o deslocamento é realizado para direita, a logica implementada é a mesma e sua representação pode ser visualizado na Fig. 3.6.

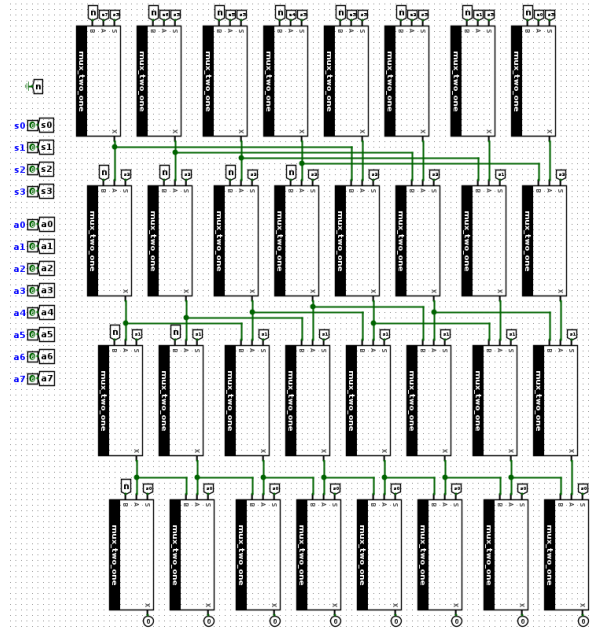


Figura 3.6: Função 9 – SHR A, B

3.1.8 Função 10 - SWP A

A função de número 10 realiza a operação de **SWP** (swap) nos quatro bits mais baixos e nos quatro bits mais altos do operando A. A saída O é obtida concatenando os bits menos significativos $A[3 : 0]$ com os bits mais significativos $A[7 : 4]$.

Essa operação permite trocar posições de grupos de 4 bits dentro de um byte. O circuito implementado consiste em conexões que reorganizam os bits de A para formar O . A representação gráfica do circuito pode ser observada na Fig. 3.7.

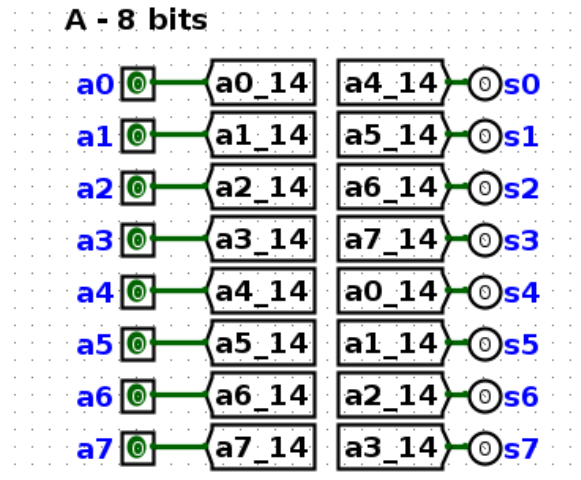


Figura 3.7: Função 10 – SWP A.

3.1.9 Função 11 - SWA A

A operação **SWA** realiza a troca de todos os bits mais significativos pelos bits menos significativos. Desta forma, é como se o a entrada binária A fosse "espelhada". A saída

O é descrita por:

$$O = A[0 : 7]$$

Esta operação troca a posição de todos os 8 bits da entrada. O circuito implementado é feito apenas redirecionando as conexões dos bits da entrada A para formar O . Por exemplo, a saída do bit 7 da entrada é conectada ao bit 0 da saída; a saída do bit 6 da entrada é conectada ao bit 1 da saída e assim sucessivamente. A representação gráfica é vista na Fig.3.8.

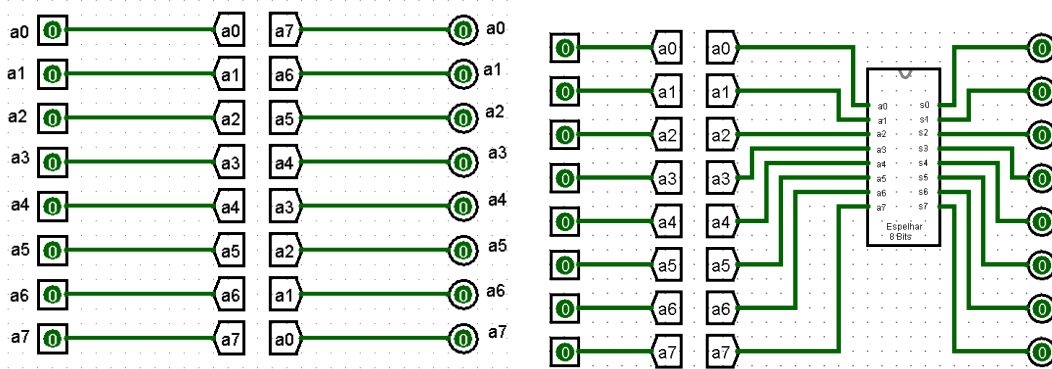


Figura 3.8: Função 11 - SWA A.

3.1.10 Função 12 - AND A,B

A função de número 12 realiza a operação lógica **AND** entre os operandos de entrada A e B . Isso significa que cada bit da saída O será igual a 1 somente se os bits correspondentes de A e B forem ambos iguais a 1; caso contrário, a saída será 0. Ela é descrita por:

$$O = A \text{ AND } B$$

O circuito implementado consiste em portas do tipo AND conectando as entradas às saídas da função. A representação gráfica do circuito pode ser observada na Fig. 3.9.

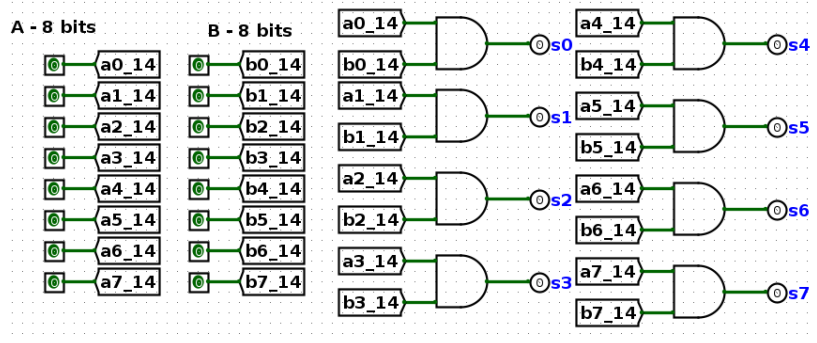


Figura 3.9: Função 12 - $O = A \text{ AND } B$.

3.1.11 Função 13 - OR A, B

A operação lógica **OR** realizada entre as entradas binárias A e B retorna 1 sempre que pelo menos um dos bit de A ou B for igual a 1. O resultado será 0 quando ambos

os bits forem 0. Ela é descrita por:

$$O = A \text{ OR } B$$

A representação gráfica é vista na Fig. 3.10.

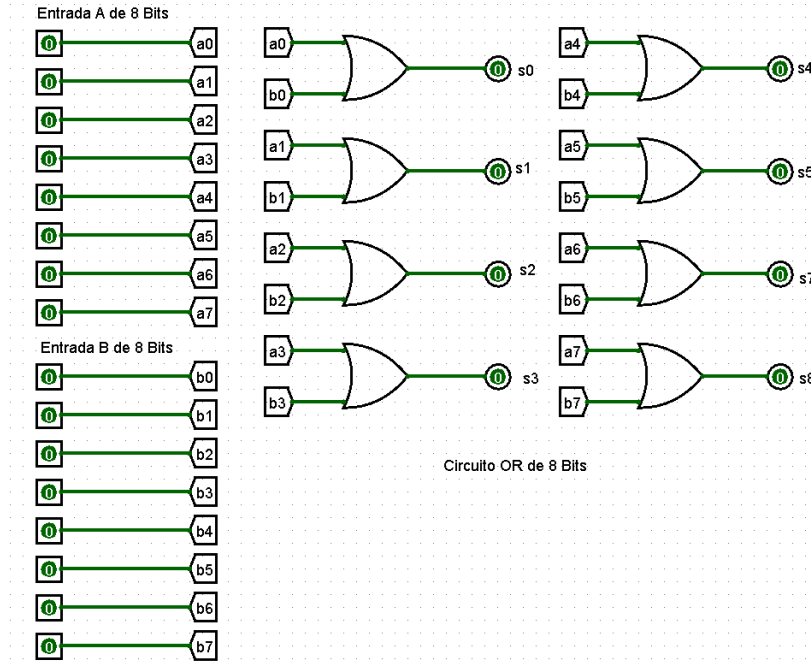


Figura 3.10: Função 13 - $O = A \text{ OR } B$.

3.1.12 Função 14 - XOR A, B

A função de número 14 realiza a operação lógica **XOR** entre os operandos de entrada A e B . Isso significa que cada bit da saída O será igual a 1 se os bits correspondentes de A e B forem diferentes, e 0 se forem iguais. Ela é descrita por:

$$O = A \text{ XOR } B$$

O circuito implementado consiste em portas do tipo XOR conectando as entradas às saídas da função. A representação gráfica do circuito pode ser observada na Fig. 3.11.

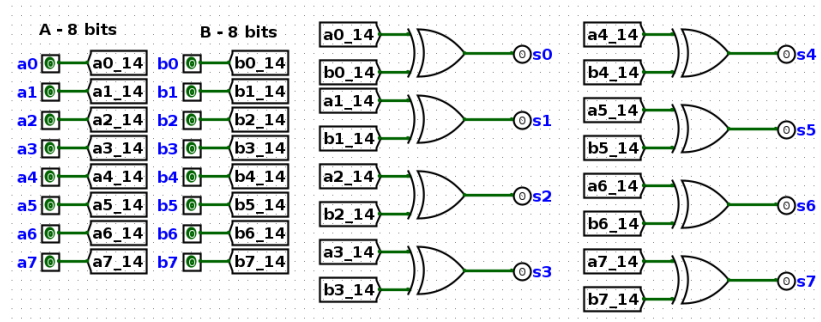


Figura 3.11: Função 14 - $O = A \text{ XOR } B$.

3.1.13 Função 15 - NOT A

A função de numero 15 realiza a negação lógica do operando de entrada A. Isso significa que cada bit do valor é invertido individualmente: bits iguais a 1 tornam-se 0 e bits iguais a 0 tornam-se 1. Ela é descrita por:

$$O = \text{not } A$$

O circuito implementado consiste em portas do tipo NOT entre as entradas e as saídas da função. A representação gráfica do circuito pode ser observado na Fig. 3.12.

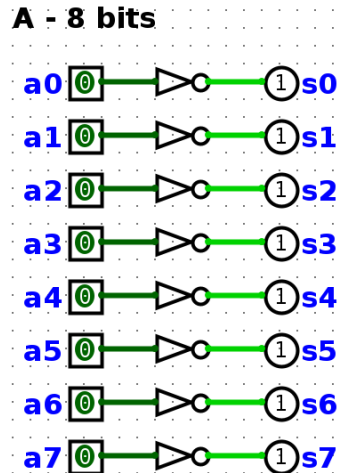


Figura 3.12: Função 15 - NOT(A).

3.1.14 Conversor BIN-BCD

O conversor **BIN-BCD** segue a mesma lógica descrita no algoritmo Double-Dabble. Para sua implementação, é necessária a utilização de uma série de circuitos auxiliares já apresentados neste relatório: um comparador de magnitude de 4 bits, um somador de 4 bits e um multiplexador 4x1.

O funcionamento é o seguinte: o comparador verifica se o valor de entrada é maior ou igual a 4 em binário. O resultado da saída “maior” do comparador é conectado à entrada de controle do multiplexador. Assim, quando essa saída está em nível lógico *HIGH*, o multiplexador direciona como saída o valor proveniente do somador; quando está em *LOW*, a saída é igual à própria entrada, conforme Fig.3.13.

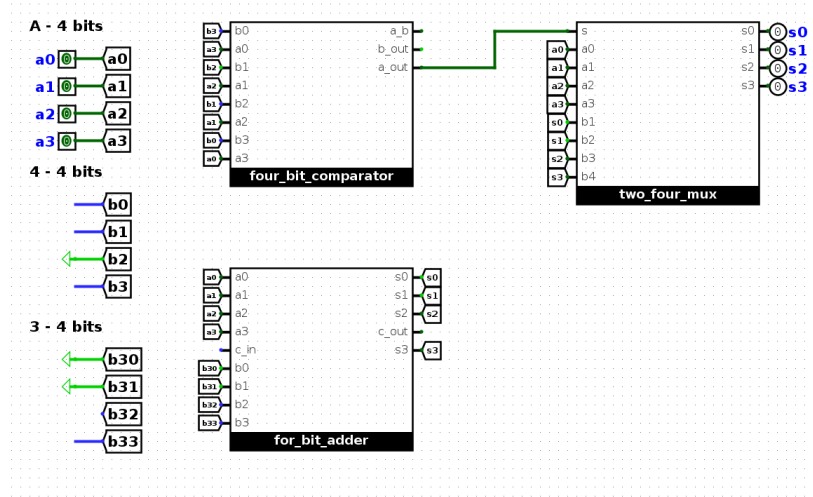


Figura 3.13: Circuito auxiliar Four or three.

Como é necessária uma quantidade substancial de arranjos compostos por esses três circuitos, foi conveniente integrá-los em um único bloco funcional, denominado *four or three*. Esses blocos são organizados em cascata, conforme ilustrado na Fig. 3.14, resultando em um comportamento equivalente ao descrito no algoritmo Double-Dabble.

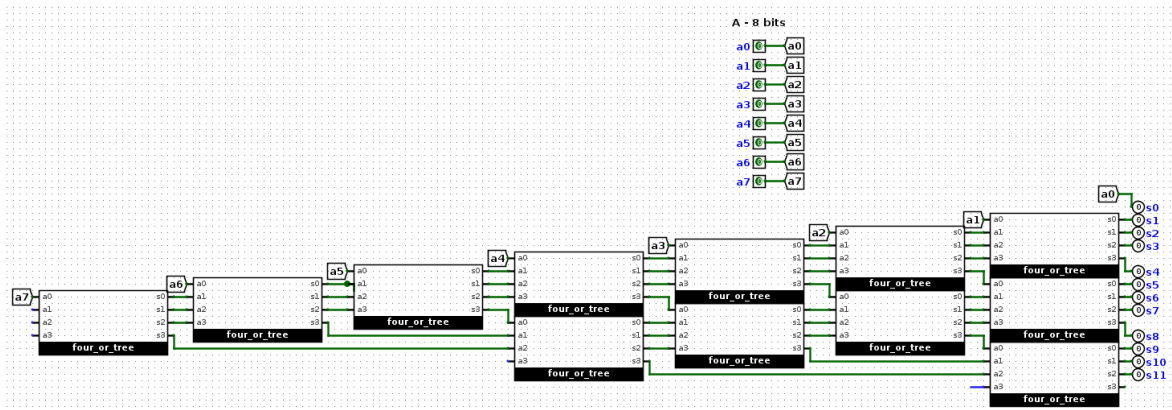


Figura 3.14: Circuito Double-Dabble.

3.1.15 Conversor BCD para 7 segmentos

Para obter a lógica do conversor **BCD para 7 segmentos**, foi elaborada a tabela verdade adotando a convenção em que as saídas correspondentes aos LEDs são nomeadas de **a** até **g**, conforme representado na Fig. 3.15.

Com base nessa convenção, construiu-se a tabela verdade do circuito e, em seguida, procedeu-se à otimização das funções lógicas por meio do mapa de Karnaugh, apresentado na Tab. 3.3.

Para essa otimização, os valores de entrada superiores a 9 foram tratados como *don't cares*, a fim de simplificar a implementação lógica.

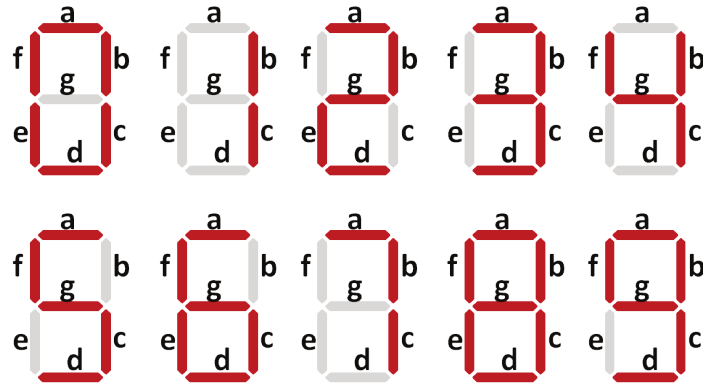


Figura 3.15: Circuito Double-Dabble.

Tabela 3.3: Tabela verdade do conversor BCD para 7 segmentos

Decimal	s0	s1	s2	s3	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1
10	1	0	1	0	x	x	x	x	x	x	x
11	1	0	1	1	x	x	x	x	x	x	x
12	1	1	0	0	x	x	x	x	x	x	x
13	1	1	0	1	x	x	x	x	x	x	x
14	1	1	1	0	x	x	x	x	x	x	x
15	1	1	1	1	x	x	x	x	x	x	x

A logica resultante a otimização por mapa de Karnaugh é a seguinte:

$$a = \overline{s0} \overline{s2} + s0 s2 + s3 + s1$$

$$b = s1 + s0 + \overline{s0} \overline{s1} + s3 + \overline{s2}$$

$$c = s3 + s2 + \overline{s1} + s0$$

$$d = s1 \overline{s0} + \overline{s2} s1 s0 + s2 \overline{s1} s0 + \overline{s2} \overline{s0}$$

$$e = \overline{s0} \overline{s2} + s1 \overline{s0}$$

$$f = s2 \overline{s1} + s1 \overline{s0} + s3 + \overline{s1} \overline{s0}$$

$$g = s2 \overline{s1} + s3 + \overline{s2} s1 + s1 \overline{s0}$$

A representação gráfica do circuito esta disponível em anexos na Fig. 3.15.

3.1.16 Multiplexador 16x9

O **multiplexador 16x9** é um circuito combinacional que tem por função selecionar, entre 16 entradas de dados distintas, uma delas para ser direcionada à saída. No caso específico do multiplexador 16x9, cada entrada possui 8 bits, e um para o bit de carry, e a seleção é realizada por meio de linhas de controle adicionais.

Para realizar a escolha entre as 16 entradas, são necessários 4 sinais de seleção, já que cada combinação binária desses sinais corresponde a uma das entradas. Assim, o multiplexador direciona exatamente um dos 16 vetores de 8 bits para a saída, de acordo com a seleção.

A sua implementação pode ser vista como a composição de oito multiplexadores 16x1, Fig. 3.16, em paralelo, onde cada multiplexador trata de um único bit das entradas de 8 bits. Dessa forma, os bits de seleção é compartilhado entre todos os blocos, garantindo que os 8 bits selecionados sejam provenientes da mesma entrada.

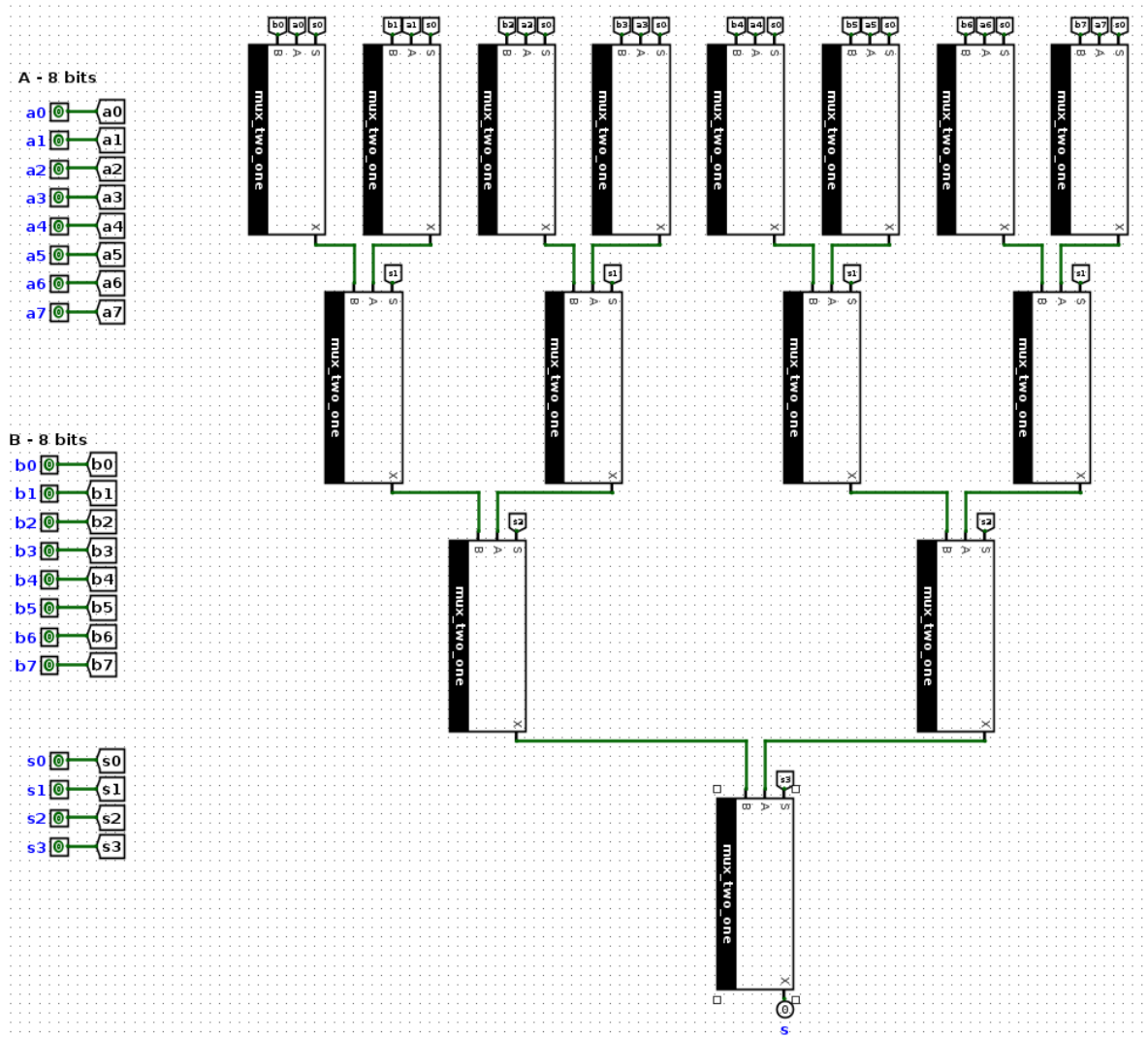


Figura 3.16: Estrutura do Multiplexador 16x1.

Esse arranjo modular e possibilita a expansão do projeto para diferentes larguras de dados. A Fig. 3.17 ilustra a estrutura do multiplexador 16x9, onde o barramento de seleção de 4 bits define qual das 16 entradas de 8 bits será propagada para a saída.

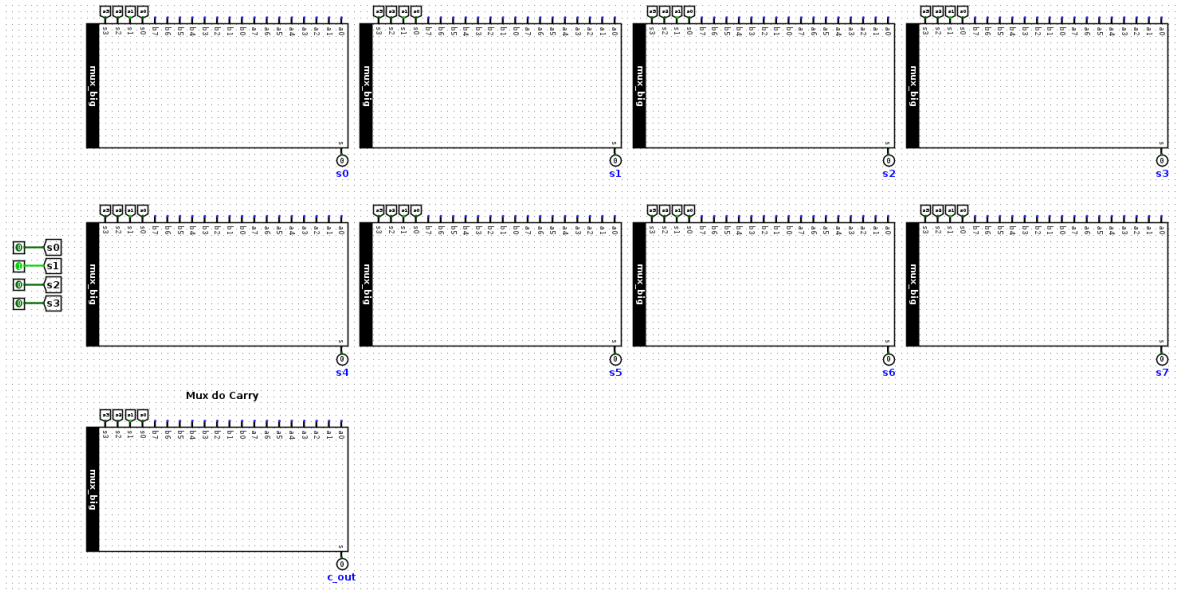


Figura 3.17: Estrutura do Multiplexador 16x9.

A Tabela 3.4 apresenta o mapeamento entre cada seleção de s_0, s_1, s_2, s_3 e a respectiva função disponibilizada na saída. Nos casos em que a combinação de seleção não corresponde a uma operação implementada, a entrada é assinalada como *não utilizada*.

Tabela 3.4: Funções selecionadas pelo decodificador de instruções

s0	s1	s2	s3	Resultado Mostrado
0	0	0	0	—
0	0	0	1	Função 1 – ADD A,B
0	0	1	0	Função 2 – SUB A,B
0	0	1	1	Função 3 – MUL A,B
0	1	0	0	Função 4 – INC A
0	1	0	1	Função 5 – DEC A
0	1	1	0	—
0	1	1	1	—
1	0	0	0	Função 8 – SHL A,B
1	0	0	1	Função 9 – SHR A,B
1	0	1	0	Função 10 – SWP A
1	0	1	1	Função 11 – SWA A
1	1	0	0	Função 12 – AND A,B
1	1	0	1	Função 13 – OR A,B
1	1	1	0	Função 14 – XOR A,B
1	1	1	1	Função 15 – NOT A

3.1.17 Comparador do Zero

O **comparador do zero** é um circuito combinacional responsável por verificar se o valor presente na saída de um bloco anterior é igual a zero. Sua função é comparar cada bit do vetor de saída e, se todos forem iguais a zero, ativar a saída de detecção.

O seu circuito, Fig. 3.18 é composto de 8 comparadores binários em cadeia, uma das entradas deve ser zero e a outra a saída resultante do Multiplexador 16x9, para o funcionamento correto.

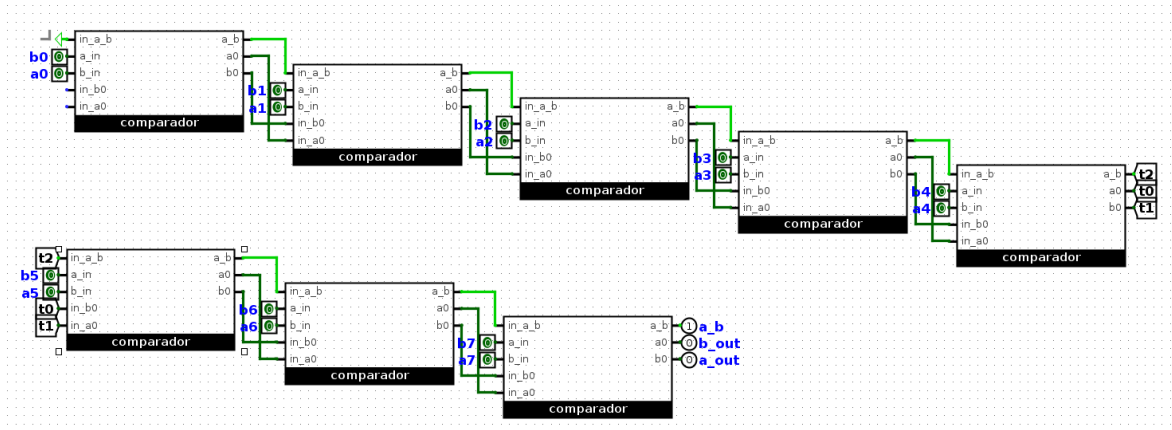


Figura 3.18: Comparador do Zero.

4 Implementação

A implementação da ULA em VHDL (VHSIC Hardware Description Language) permite sintetizar em hardware o projeto lógico. Nele, as portas lógicas são representadas por:

Tabela 4.1: Representação das portas lógicas em VHDL

Porta Lógica	Operador em VHDL
AND	<code>and</code>
OR	<code>or</code>
NOT	<code>not</code>
NAND	<code>not and</code>
NOR	<code>not or</code>
XOR	<code>xor</code>
XNOR	<code>xnor</code>

A estrutura de um código em VHDL é organizada em dois blocos principais: a entidade e a arquitetura. A entidade define a interface do componente, ou seja, as portas de entrada e saída, seus tipos e direções. A arquitetura descreve o funcionamento interno do componente, podendo ter outros componentes que fazem parte do mesmo. Essa divisão garante modularidade no desenvolvimento do sistema, facilitando a programação.

Para a programação do projeto, foi utilizado o software ModelSim, que permite programar e simular de circuitos descritos em VHDL. Com ele, é possível realizar a compilação dos módulos, gerar sinais de teste e observar as formas de onda correspondentes às saídas. O processo de simulação garante que os componentes implementados correspondam ao comportamento esperado antes da implementação em hardware.

A seguir, iremos detalhar como o grupo projetou a implementação de cada etapa da ULA.

4.0.1 Função 1 - ADD A,B

Este circuito é construído seguindo uma arquitetura modular, onde a funcionalidade principal é obtida através da interconexão de oito somadores de 1 bit, componentes denominados *Somador1Bit* criando assim um somador completo.

A entidade *Somador8Bits* possui cinco portas principais que definem sua interface externa. As entradas A e B são vetores de 8 bits do tipo *std_logic_vector(7 downto 0)* que representam os operandos da soma. A entrada Cin corresponde ao carry de entrada

inicial, permitindo que o somador seja utilizado em cascata com outros somadores ou para implementar operações que necessitem de um carry inicial. As saídas incluem S, um vetor de 8 bits que contém o resultado da soma, e Cout, que representa o carry de saída final da operação.

A arquitetura denominada *CKT* implementa o somador através de uma abordagem hierárquica, utilizando o componente *Somador8Bits* como bloco construtivo fundamental. Este componente representa um somador completo de 1 bit que recebe dois bits de entrada (A e B), um carry de entrada (Cin), e produz um bit de soma (S) e um carry de saída (Cout).

Para implementar a propagação de carry entre os bits adjacentes, foram declarados sete sinais internos (C1, C2, C3, C4, C5, C6, C7) que transportam os valores de carry entre as etapas sucessivas da soma. Esta propagação sequencial do carry é característica dos somadores do tipo ripple-carry, onde o carry gerado por cada bit é utilizado como entrada para o bit seguinte na ordem de significância.

4.0.2 Função 2 - SUB A,B

A entidade *Subtrator8Bits* é projetada com uma arquitetura modular que reutiliza como componente *Somador1Bit*. Esta implementação converte internamente a operação de subtração em uma operação de adição, aproveitando o fato matemático de que $A - B$ é equivalente a $A + (-B)$, onde $-B$ é representado pelo complemento de dois de B.

A interface da entidade define claramente os sinais de entrada e saída necessários para a operação de subtração. Os sinais de entrada A e B são vetores de 8 bits que representam os operandos da subtração, enquanto o sinal de saída S fornece o resultado da operação também em 8 bits. O sinal Cout representa o carry-out final da operação, que em contexto de subtração pode indicar se houve um empréstimo (borrow) na operação. Um aspecto interessante da implementação é a presença do sinal buffer Bn, que armazena o complemento bit a bit de B, tornando visível externamente essa etapa intermediária do processo de cálculo do complemento de dois.

A estratégia adotada para realizar a subtração baseia-se na conversão do operando B para seu complemento de dois através de duas etapas distintas. Primeiro, cada bit de B é invertido individualmente através de operações NOT, gerando o complemento bit a bit que é armazenado no vetor Bn. Esta inversão é realizada de forma explícita para cada posição do vetor, desde Bn(0) até Bn(7). Posteriormente, a adição de 1 ao resultado invertido é realizada implicitamente através do sinal Cin_internal, que é inicializado com valor lógico '1' e conectado ao carry-in do primeiro somador da cadeia.

A implementação utiliza oito instâncias do componente *Somador1Bit* organizadas em uma configuração de ripple-carry, onde o carry-out de cada estágio torna-se o carry-in do próximo estágio. Esta arquitetura cascata permite que a operação de adição (e consequentemente a subtração) seja propagada bit a bit. Os sinais internos C1 através C7 transportam os carries intermediários entre os estágios consecutivos, garantindo que o resultado final seja matematicamente correto. O primeiro estágio (U0) recebe o Cin_internal com valor '1', implementando assim a adição necessária para completar o complemento de dois.

4.0.3 Função 3 - MUL A,B

O multiplicador de 8 bits utiliza o algoritmo de multiplicação por soma e deslocamento (shift-and-add). Ele usa abordagem estrutural para realizar multiplicação de números binários, onde apenas os 8 bits menos significativos do resultado são retornados como saída final.

A entidade *mult_8bit* define dois operandos de entrada *a* e *b* de 8 bits cada, uma saída *s* de 8 bits que representa o resultado da multiplicação, e uma saída *c_out* para indicar carry out. A arquitetura interna utiliza dois componentes principais: o *and_64bit* para gerar os produtos parciais e o *adder_16bit* para realizar as somas acumulativas necessárias.

O componente *and_64bit* é responsável pela geração dos produtos parciais da multiplicação através de operações AND bit a bit. Ele recebe os dois operandos de 8 bits e produz 8 vetores de saída 's0' até 's7', onde cada vetor representa o produto do operando *a* com um bit específico do operando *b*. Este processo segue o princípio fundamental da multiplicação binária, onde cada bit do multiplicador gera um produto parcial que deve ser adequadamente posicionado e somado para obter o resultado final.

A soma dos produtos parciais é realizada através de uma cascata de sete somadores de 16 bits (*adder_16bit*). Cada somador na sequência adiciona um novo produto parcial ao resultado acumulado do somador anterior, implementando o deslocamento necessário através do mapeamento dos sinais de entrada, os sinais internos foram 7 vetores de 16 bits, *t0* a *t6*, e 8 vetores de 8 bits, *temp0* a *temp7*. O primeiro somador *U0* combina os produtos parciais *temp0* e *temp1*, onde *temp1* é deslocado uma posição à esquerda. Os somadores subsequentes (*U1* a *U6*) continuam este processo, cada um adicionando o próximo produto parcial com o deslocamento apropriado.

A saída é composta por 9 bits, sendo os 8 menos significativos correspondentes ao resultado que será encaminhado à ULA, e 1 bit adicional de *carry out*. Esse bit de *carry out* é obtido por meio da operação lógica OR aplicada entre os 8 bits mais significativos. As vantagens desta implementação incluem sua estrutura modular, que permite reutilização dos componentes *adder_16bit* e facilita modificações futuras.

4.0.4 Função 4 - INC A

A entidade *Incrementador8Bits* recebe o vetor *A*, composto por 8 bits, e gera como saída o vetor *S*, também de 8 bits, que corresponde ao valor incrementado. Além disso, existe a saída *Cout*, que representa o bit de carry caso ocorra um transbordamento (overflow) após a operação.

A arquitetura do circuito foi construída a partir da utilização do componente *Somador1Bits* que realiza a soma de apenas um bit da entrada com um bit adicional e o sinal de carry proveniente da etapa anterior. Para formar o incrementador completo, foram interligados oito desses somadores, um para cada posição do vetor.

O funcionamento segue a lógica de um somador ripple-carry: na primeira posição (bit menos significativo), o valor de entrada *A(0)* é somado com zero e com um carry interno inicializado em '1'. Isso equivale a realizar a operação $A + 1$. Os demais bits são somados apenas com o carry resultante da etapa anterior, propagando o efeito do incremento ao longo de todo o vetor. Caso todos os bits da entrada sejam iguais a 1, o resultado será 00000000 e o sinal *Cout* será ativado, indicando o transbordamento.

4.0.5 Função 5 - DEC A

O módulo *Decrementador8Bits* realiza a operação de decremento em um número binário de 8 bits. A implementação utiliza uma abordagem baseada em complemento de dois, onde a operação $A - 1$ é realizada através da soma $A + 11111111$, aproveitando a aritmética binária para obter o resultado desejado.

A arquitetura do decrementador é construída utilizando oito componentes *Somador1Bits*. Cada somador de 1 bit recebe como entradas o bit correspondente do número de entrada A , o valor '1', e o carry proveniente do bit anterior. Esta configuração permite que o circuito execute efetivamente a subtração através da adição do complemento.

O módulo possui três portas principais: a entrada A de 8 bits, a saída S de 8 bits que representa o resultado do decremento, e a saída Cout que indica o carry-out final da operação. Internamente, são utilizados sete sinais intermediários (C1 a C7) para propagação do carry entre os somadores. O carry inicial (Cin_internal) é configurado como '0', iniciando a cadeia de soma a partir do bit menos significativo.

4.0.6 Função 8 - SHL A,B

A entidade *shl_8bit* implementa um deslocador lógico à esquerda de 8 bits. Ele desloca a entrada A de acordo com o número formado pelos 3 bits menos significativos de B . A função possui três portas principais: o vetor de entrada e de 8 bits (7 downto 0), o sinal de controle s de 3 bits (2 downto 0) que determina a quantidade de deslocamento, e o vetor de saída z de 8 bits (7 downto 0) que contém o resultado da operação. O sinal de controle de 3 bits permite especificar deslocamentos de 0 a 7 posições, cobrindo toda a faixa possível para um dado de 8 bits.

A implementação utiliza uma arquitetura hierárquica baseada em multiplexadores 2x1, organizados em três estágios distintos. Cada estágio é responsável por um bit específico do sinal de controle: o primeiro estágio (UI30-UI37) é controlado pelo bit mais significativo $s(2)$ e implementa deslocamentos de 0 ou 4 posições, o segundo estágio (UI20-UI27) é controlado por $s(1)$ e adiciona deslocamentos de 0 ou 2 posições, e o terceiro estágio (UI10-UI17) é controlado por $s(0)$ e adiciona deslocamentos de 0 ou 1 posição. Esta estrutura em cascata permite que qualquer combinação dos bits de controle resulte no deslocamento desejado, desde 000 (nenhum deslocamento) até 111 (deslocamento de 7 posições).

No primeiro estágio, quando $s(2)$ é '0', os dados passam diretamente através dos multiplexadores sem alteração. Quando $s(2)$ é '1', os dados são deslocados 4 posições para a esquerda, onde os 4 bits mais significativos são perdidos e os 4 bits menos significativos recebem zeros. O segundo estágio opera de forma similar, mas com deslocamentos de 2 posições: quando $s(1)$ é '0', os dados do estágio anterior passam inalterados, e quando $s(1)$ é '1', ocorre um deslocamento adicional de 2 posições para a esquerda. O terceiro e último estágio implementa o deslocamento de 1 posição, controlado por $s(0)$.

A função utiliza dois sinais intermediários, $t2$ e $t1$, ambos vetores de 8 bits, para armazenar os resultados parciais entre os estágios. O sinal $t2$ contém a saída do primeiro estágio (multiplexadores UI30-UI37), enquanto $t1$ armazena a saída do segundo estágio (multiplexadores UI20-UI27). Esta abordagem permite uma implementação modular.

Para o tratamento do *carry*, foi necessário adicionar um componente multiplexador extra em cada estágio. Esse componente verifica os bits deslocados em cada operação

parcial e, ao final, os três sinais de *carry* gerados são combinados por meio de uma porta OR, resultando no sinal único de *carry out*.

4.0.7 Função 9 - SHR A,B

O circuito implementado corresponde a um deslocador à direita de 8 bits, utilizando uma arquitetura baseada em multiplexadores. A entidade *shr_8bit* recebe como entrada um vetor de 8 bits *e*, um sinal de controle de 3 bits *s* que determina a quantidade de deslocamento, e produz como saída um vetor de 8 bits *z* com o resultado do deslocamento à direita.

A arquitetura do circuito utiliza exclusivamente componentes mux2x1 (multiplexadores 2 para 1), organizados em três estágios hierárquicos que permitem realizar deslocamentos de 0 a 7 posições. Esta implementação segue o princípio de decomposição binária, onde cada bit do sinal de controle (*s*) corresponde a um estágio específico: *s*(2) controla deslocamentos de 4 posições, *s*(1) controla deslocamentos de 2 posições, e *s*(0) controla deslocamentos de 1 posição. A combinação destes três bits permite obter qualquer valor de deslocamento entre 0 e 7.

No primeiro estágio, quando *s*(2) é '0', os dados passam diretamente através dos multiplexadores sem alteração. Quando *s*(2) é '1', os dados são deslocados 4 posições para a direita, onde os 4 bits mais significativos são perdidos e os 4 bits menos significativos recebem zeros. O segundo estágio opera de forma similar, mas com deslocamentos de 2 posições: quando *s*(1) é '0', os dados do estágio anterior passam inalterados, e quando *s*(1) é '1', ocorre um deslocamento adicional de 2 posições para a direita. O terceiro e último estágio implementa o deslocamento de 1 posição, controlado por *s*(0),

4.0.8 Função 10 - SWP A

A entidade *SWP* realiza uma operação de troca entre os nibbles (4 bits) de um vetor de 8 bits. A entrada, representada pelo sinal *A*, é um byte que é dividido em duas partes: os quatro bits mais significativos e os quatro bits menos significativos. O circuito descrito reorganiza esses valores de forma que os bits mais altos passem a ocupar as posições mais baixas da saída *Y*, enquanto os bits mais baixos passam a ocupar as posições mais altas. Dessa maneira, ocorre uma permutação completa entre os nibbles da palavra de entrada.

4.0.9 Função 11 - SWA A

A entidade *SWA A* implementa uma operação de inversão de bits em um vetor de 8 posições. A entrada, representada pelo sinal *A*, é um byte que é reorganizado de forma espelhada na saída *Y*. Isso significa que o bit mais significativo de *A* (*A*7) passa a ocupar a posição menos significativa em *Y* (*Y*0), enquanto o bit menos significativo (*A*0) passa a ocupar a posição mais significativa (*Y*7). Assim, toda a entrada é invertida em ordem, resultando em uma saída que corresponde ao valor binário de *A* lido de trás para frente.

4.0.10 Função 12 - AND A,B

Essa função implementa uma operação lógica AND bit entre dois vetores de 8 bits. As entradas *A* e *B* representam dois operandos binários do tipo `std_logic_vector(7 downto 0)`, e a saída *Y* armazena o resultado da operação. O comportamento do circuito consiste em comparar cada bit correspondente de *A* e *B*, de forma que *Y* só assume valor '1' em uma determinada posição quando ambos os bits de entrada também forem '1'; caso contrário, a saída nessa posição será '0'.

4.0.11 Função 13 - OR A, B

Essa função implementa uma porta lógica OR de 8 bits entre dois vetores de 8 bits, *A* e *B*, e gerar um vetor de 8 bits como saída, *Y*, que contém o resultado da operação, a arquitetura é definida por meio do nome `porta_or`, onde é implementada a operação lógica OR entre os vetores *A* e *B*, que realiza a operação bit a bit entre os bits correspondentes de cada vetor de entrada. O resultado dessa operação é atribuído diretamente ao vetor de saída *Y*.

4.0.12 Função 14 - XOR A, B

Essa função descreve de uma porta lógica XOR de 8 bits, denominada *Pxor*. O propósito desta porta é realizar a operação lógica XOR (exclusive OR) entre dois vetores de 8 bits, *A* e *B*, e gerar um vetor de 8 bits como saída, *Y*, que contém o resultado dessa operação. A operação XOR é uma operação lógica que resulta em 1 apenas quando os bits de entrada são diferentes (ou seja, um é 1 e o outro é 0), e resulta em 0 quando os bits são iguais.

Na arquitetura, a operação XOR é realizada entre os vetores *A* e *B* utilizando o operador `xor`, que realiza a operação exclusive OR bit a bit entre os dois vetores de entrada com a seguinte tabela de verdade: Se os bits de *A* e *B* forem iguais (ambos 0 ou ambos 1), o resultado será 0. Se os bits de *A* e *B* forem diferentes (um é 0 e o outro é 1), o resultado será 1

4.0.13 Função 15 - NOT A

O objetivo dessa função é realizar a operação lógica NOT em um vetor de 8 bits de entrada, *A*, e gerar um vetor de 8 bits como saída, *Y*, que conterá o resultado da inversão bit a bit de *A*. Na arquitetura do componente, a operação lógica NOT é aplicada ao vetor de entrada *A*. A linha `Y(7 downto 0) <= not A(7 downto 0)`; faz uso do operador `not`, que inverte cada bit do vetor de entrada *A*. Ou seja, para cada bit de *A*, se o valor for 0, ele será invertido para 1, e se o valor for 1, ele será invertido para 0. O resultado dessa operação é armazenado no vetor de saída *Y*.

4.0.14 Conversor BDC para 7 segmentos

No projeto foi implementado um decodificador BCD para display de sete segmentos. A entidade definida recebe como entrada um vetor de 4 bits (*s*), que representa um número em código BCD (Binary Coded Decimal), e fornece como saída um vetor de 7

bits (z), que controla diretamente os segmentos do display. Cada posição do vetor de saída ($z(0)$ até $z(6)$) corresponde a um segmento do display, e o valor lógico atribuído determina se o segmento estará aceso ou apagado.

Na arquitetura descrita, cada saída é formada por expressões lógicas envolvendo os bits de entrada ($s(3)$, $s(2)$, $s(1)$, $s(0)$). Essas expressões foram elaboradas a partir do mapa de Karnaugh ou das tabelas de verdade que associam cada dígito decimal (0–9) à configuração necessária dos segmentos do display. Dessa forma, quando um número decimal é fornecido na entrada, o circuito gera automaticamente o padrão correto para representar esse número no display de sete segmentos.

4.0.15 Double Dabble

O algoritmo **Double Dabble** é responsável pela conversão de números binários em representação decimal codificada em BCD. A entidade definida recebe como entrada um vetor binário de largura fixa, correspondente ao valor a ser convertido, e fornece como saída um conjunto de dígitos BCD que podem ser utilizados para exibição ou processamento posterior.

Na arquitetura descrita, a conversão é realizada de forma iterativa, por meio de deslocamentos sucessivos à esquerda do valor binário, com a inserção progressiva dos bits de maior peso nos registradores de saída. A cada iteração, os grupos parciais de 4 bits são avaliados: caso algum deles assuma valor maior ou igual a 5, é adicionada a constante “3” ao grupo, garantindo a correção da representação decimal. Esse procedimento é realizado por um componente chamado *four or three*, esse é repetido até todos os bits sejam processados.

4.0.16 Multiplexador 16x9

O componente BIGMUX representa um multiplexador de grande escala, especificamente projetado para selecionar entre 16 entradas diferentes, onde cada entrada consiste em um vetor de 8 bits. Esta estrutura é particularmente útil em sistemas digitais complexos, como unidades lógicas e aritméticas (ULA), onde múltiplas operações podem gerar resultados de 8 bits e é necessário selecionar qual resultado será direcionado para a saída final do sistema.

A arquitetura do BIGMUX utiliza uma abordagem modular baseada na instanciação de múltiplos componentes MUX16X1, o BIGMUX instancia exatamente 8 componentes MUX16X1, um para cada bit das entradas e saída de 8 bits.

A implementação utiliza um comando "generate" do VHDL para criar as 8 instâncias do MUX16X1 de forma iterativa. Cada instância do MUX16X1 é responsável por selecionar um bit específico (posição i) de todas as 16 entradas de 8 bits, utilizando o mesmo sinal de seleção S de 4 bits.

Apesar de o MUX 16x8 possuir 16 entradas, somente 13 são necessárias para as operações da ULA. As 3 entradas excedentes serão atribuídas ao valor lógico ‘0’.

O sinal de seleção S , composto por 4 bits, permite a escolha entre as 16 entradas possíveis ($2^4 = 16$), representando as combinações binárias de 0000 a 1111. Quando $S = "0000"$, a saída Z receberá o valor de I_0 ; quando $S = "0001"$, receberá I_1 , e assim sucessivamente até $S = "1111"$ para I_{15} .

A estrutura de portas do componente segue padrões convencionais de VHDL, com todas as entradas definidas como vetores de 8 bits do tipo `std_logic_vector(7 downto 0)`, enquanto a saída Z mantém o mesmo formato. Esta consistência facilita a integração do componente em sistemas maiores e garante compatibilidade com outros módulos que operem com dados de 8 bits.

Adicionalmente, foi implementado um MUX 16x1 destinado ao direcionamento do sinal de carry associado à função em execução. A Figura 4.1 apresenta um excerto do código VHDL empregado na geração do BIGMUX, contemplando tanto o MUX16x9, responsável pela saída de 8 bits, quanto o MUX16x1, correspondente à saída do carry.

```
BEGIN
  -- 8 muxes para os bits do resultado
  gen_mux : FOR i IN 0 TO 7 GENERATE
    U : MUX16X1 PORT MAP(
      I0(i), I1(i), I2(i), I3(i), I4(i), I5(i), I6(i), I7(i),
      I8(i), I9(i), I10(i), I11(i), I12(i),
      '0', '0', '0', -- entradas extras zero
      S, Z(i)
    );
  END GENERATE;

  -- 1 mux para o carry (somente 6 funcoes que tem carry)
  Ucarry : MUX16X1 PORT MAP(
    C0, C1, C2, C3, C4, C5, C6,
    '0', '0', '0', '0', '0', '0', '0', '0', '0', -- entradas extras zero
    S, C
  );
```

Figura 4.1: Descrição da saída do BIGMUX em VHDL.

Em termos de aplicação prática, este multiplexador pode ser utilizado como elemento de seleção final em uma ULA, onde cada entrada I0 a I15 representaria o resultado de uma operação específica (soma, subtração, AND, OR, XOR, deslocamentos, etc.). O sinal de seleção S seria então controlado pela unidade de controle do processador, determinando qual operação deve ser apresentada na saída da ULA baseada na instrução sendo executada.

4.0.17 Correções do projeto

Comparador do Zero

O **comparador do zero** foi otimizado para uma arquitetura simples. Sua lógica de funcionamento baseia-se na aplicação de uma operação OR entre todos os bits de entrada. O resultado dessa operação é, em seguida, invertido por uma porta NOT.

Dessa forma, a saída do comparador assume nível lógico alto (1) somente quando todas as entradas forem iguais a zero, indicando que o valor analisado é nulo. Essa abordagem minimiza a complexidade do circuito anterior, uma vez que utiliza apenas portas OR e uma inversora, mantendo baixo o custo de implementação e a área ocupada em hardware, o código está disponível em anexos, na Fig. C.4.

Otimização do four or three

De modo a reduzir a complexidade no algoritmo *Double-Dabble*, buscou-se a otimização do componente *four or three*, o qual é instanciado 12 vezes na arquitetura. Para isso, elaborou-se a tabela verdade correspondente (Tab. 4.2) e aplicou-se o processo de minimização por meio de mapas de Karnaugh. As expressões lógicas resultantes foram:

$$s_0 = \overline{a_0} a_3 + \overline{a_0} a_1 a_2 + a_0 \overline{a_2} \overline{a_3}$$

$$s_1 = \overline{a_0} \overline{a_1} a_3 + a_1 \overline{a_2} \overline{a_3} + a_0 a_1$$

$$s_2 = \overline{a_0} \overline{a_1} a_2 + a_1 \overline{a_2} a_3 + a_0 \overline{a_2} a_3$$

$$s_3 = \overline{a_0} \overline{a_1} a_3 + \overline{a_2} a_3 + a_1 a_2 \overline{a_3} + a_0 a_2 \overline{a_3}$$

Tabela 4.2: Tabela verdade four or three

a0	a1	a2	a3	s0	s1	s2	s3
0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	1
0	0	1	0	1	0	1	0
0	0	1	1	1	1	1	0
0	1	0	0	0	1	0	1
0	1	0	1	0	0	1	1
0	1	1	0	1	1	0	0
0	1	1	1	0	0	0	1
1	0	0	0	1	0	0	1
1	0	0	1	0	1	1	0
1	0	1	0	0	0	0	1
1	0	1	1	1	1	0	0
1	1	0	0	1	0	1	1
1	1	0	1	0	1	0	0
1	1	1	0	1	0	1	0
1	1	1	1	0	1	0	0

O código em VHDL está disponível em anexos, na Fig. C.5.

4.0.18 Montagem do Sistema

Com o objetivo de melhorar a organização e a modularidade do código VHDL, optou-se por dividir a implementação da ULA em dois blocos principais, conforme Fig. 4.2.

O primeiro bloco, denominado **Bloco de Funções**, contém todas as operações lógicas e aritméticas executadas em paralelo, além do multiplexador 16x9 responsável por selecionar a saída correspondente à função desejada.

O segundo bloco reúne os módulos auxiliares de conversão, compostos pelo **Conversor binário para BCD** (implementado pelo algoritmo Double-Dabble) e pelo **Con-**

versor BCD para sete segmentos, responsável por gerar o padrão de ativação dos LEDs do display.

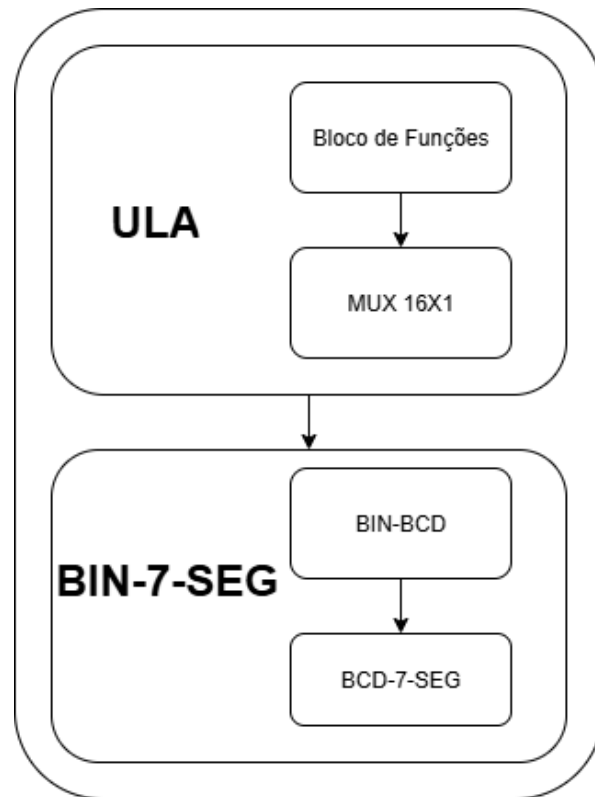


Figura 4.2: Estrutura do Código.

5 Resultados

Nesta seção são apresentados os resultados obtidos a partir da implementação e simulação dos blocos desenvolvidos. O objetivo é verificar se o comportamento descrito em código VHDL corresponde às especificações funcionais definidas no projeto. São mostradas as formas de onda de simulação, que permitem validar o correto funcionamento dos componentes.

5.1 Bloco de Funções

O **bloco de funções** foi implementado como uma entidade VHDL denominada *BLOCOFUNCOES*, Fig. 5.2, responsável por agrupar em paralelo todas as operações lógicas e aritméticas da ULA. Esse bloco recebe como entradas dois vetores de 8 bits (A e B) e disponibiliza diversas saídas correspondentes às funções implementadas, além dos sinais de *carry out* quando aplicável.

Cada uma das funções é instanciada como um componente dentro da arquitetura *CKT*, de forma que todos os resultados são processados em paralelo. A seleção da operação desejada é realizada posteriormente pelo multiplexador 16x9, que direciona a saída correspondente para a ULA.

A entidade *ULA_P1* representa a integração dos blocos funcionais que compõem a unidade lógica e aritmética. Ela recebe como entradas dois vetores de 8 bits (A e B), que são os operandos, e um vetor de 4 bits (S_{in}), responsável pela seleção da operação desejada. Como saída, a ULA disponibiliza um vetor de 8 bits (S_{out}), além dos sinais de *carry out* (C_{out}) e *zero* (Z_{out}). Na arquitetura, três componentes principais são instanciados:

- **BLOCOFUNCOES**: responsável pela execução em paralelo de todas as operações aritméticas e lógicas disponíveis. Cada função gera sua respectiva saída e, quando aplicável, também um sinal de *carry*.
- **BIGMUX**: multiplexador de 16 entradas por 9 saídas, utilizado para selecionar, com base em S_{in} , qual das operações geradas no *BLOCOFUNCOES* será propagada para a saída da ULA. Esse módulo também direciona o *carry out* correspondente.
- **comparador_zero**: verifica se o resultado selecionado pelo multiplexador é nulo. Caso todos os bits de saída sejam zero, o sinal Z_{out} é ativado.

Assim, o módulo *ULA_P1* integra os resultados de todos os blocos internos, encaminhando à saída apenas a função escolhida pela palavra de controle, juntamente com os indicadores de estado.

5.2 Conversor BIN-7-SEG

O módulo *bin_7_seg* implementa um sistema completo para converter um número binário de 8 bits em sinais de controle para três displays de 7 segmentos, permitindo a exibição decimal do valor binário de entrada que pode variar de 0 a 255. A interface do módulo é composta por uma entrada *s*, que consiste em um vetor de 8 bits representando o valor binário a ser convertido, e sete saídas denominadas *a*, *b*, *c*, *d*, *e*, *f* e *g*.

O funcionamento do sistema segue um fluxo de dados bem definido. Inicialmente, o valor binário de entrada é enviado ao componente *double_dabble* (instanciado como U1), que realiza a conversão para três dígitos BCD armazenados nos sinais intermediários *a_carry*, *b_carry* e *c_carry*. Estes sinais transportam respectivamente os valores das unidades, dezenas e centenas do número decimal. Subsequentemente, cada dígito BCD é processado por uma instância separada do componente *bcd_7seg*: a instância U2 converte as unidades para controlar o primeiro display, U3 converte as dezenas para o segundo display, e U4 converte as centenas para o terceiro display.

O mapeamento dos sinais de saída segue a convenção padrão de displays de 7 segmentos, onde cada segmento (*a* até *g*) é representado por um vetor de 3 bits. A conexão entre os componentes *bcd_7seg* e as saídas finais é realizada de forma que cada bit do vetor de saída *z* de cada instância *bcd_7seg* é mapeado para o bit correspondente do segmento apropriado na saída final. Esta abordagem garante que os três displays possam ser controlados simultaneamente com a representação decimal correta do valor binário de entrada.

A Fig. 5.1 ilustra o funcionamento desse bloco. Como exemplo, considera-se a entrada binária $[0, 0, 0, 0, 1, 1, 0, 0]$, que representa o valor decimal 12. Após a conversão, a saída em BCD é mapeada para os vetores $[1, 1, 1, 1, 1, 1, 1]$, $[0, 1, 1, 0, 0, 0, 0]$ e $[1, 1, 0, 1, 1, 0, 1]$, os quais correspondem, respectivamente, aos dígitos decimais 0, 1 e 2, respectivamente. Essa representação segue a convenção estabelecida no Tópico 3.1.15.

5.3 Gravação e testes

O mapeamento dos pinos seguiu conforme a Tab. 5.1, buscando facilitar a utilização da ULA ao organizar os sinais de entrada de forma sequencial e próxima no hardware.

Após a síntese, compilação e gravação do programa em VHDL, foram identificados alguns ajustes necessários relacionados ao comportamento da placa utilizada. Observou-se que os botões, quando não pressionados, permaneciam em nível lógico alto (*HIGH*). Como esse detalhe não havia sido considerado inicialmente, foi necessário corrigir as entradas invertendo a lógica de acionamento.

Outro ajuste foi necessário no display de sete segmentos. Durante os testes, verificou-se que o estado de ativação dos LEDs correspondia ao nível lógico baixo (*LOW*), diferindo da implementação inicial, que assumia nível lógico alto. Dessa forma, também foi preciso inverter a lógica de controle dos segmentos do display.

Com esses ajustes realizados, a implementação funcionou corretamente, e todas as funções se comportaram conforme as simulações realizadas previamente no software ModelSim. A Fig. 5.4 ilustra a FPGA utilizada para os testes.

Tabela 5.1: Mapeamento dos pinos da FPGA

Switch	Pino	Descrição	Referencia
SW[0]	PIN_N25	Toggle Switch 0	A[0]
SW[1]	PIN_N26	Toggle Switch 1	A[1]
SW[2]	PIN_P25	Toggle Switch 2	A[2]
SW[3]	PIN_AE14	Toggle Switch 3	A[3]
SW[4]	PIN_AF14	Toggle Switch 4	A[4]
SW[5]	PIN_AD13	Toggle Switch 5	A[5]
SW[6]	PIN_AC13	Toggle Switch 6	A[6]
SW[7]	PIN_C13	Toggle Switch 7	A[7]
SW[8]	PIN_B13	Toggle Switch 8	B[0]
SW[9]	PIN_A13	Toggle Switch 9	B[1]
SW[10]	PIN_N1	Toggle Switch 10	B[2]
SW[11]	PIN_P1	Toggle Switch 11	B[3]
SW[12]	PIN_P2	Toggle Switch 12	B[4]
SW[13]	PIN_T7	Toggle Switch 13	B[5]
SW[14]	PIN_U3	Toggle Switch 14	B[6]
SW[15]	PIN_U4	Toggle Switch 15	B[7]
KEY[0]	PIN_G26	Pushbutton 0	S[0]
KEY[1]	PIN_N23	Pushbutton 1	S[1]
KEY[2]	PIN_P23	Pushbutton 2	S[2]
KEY[3]	PIN_W26	Pushbutton 3	S[3]
LEDR[0]	PIN_AE23	LED Red[0]	C_out
LEDR[1]	PIN_AF23	LED Red[1]	Z_out
HEX0[0]	PIN_AF10	Seven Segment Digit 0[0]	a[0]
HEX0[1]	PIN_AB12	Seven Segment Digit 0[1]	b[0]
HEX0[2]	PIN_AC12	Seven Segment Digit 0[2]	c[0]
HEX0[3]	PIN_AD11	Seven Segment Digit 0[3]	d[0]
HEX0[4]	PIN_AE11	Seven Segment Digit 0[4]	e[0]
HEX0[5]	PIN_V14	Seven Segment Digit 0[5]	f[0]
HEX0[6]	PIN_V13	Seven Segment Digit 0[6]	g[0]
HEX1[0]	PIN_V20	Seven Segment Digit 1[0]	a[1]
HEX1[1]	PIN_V21	Seven Segment Digit 1[1]	b[1]
HEX1[2]	PIN_W21	Seven Segment Digit 1[2]	c[1]
HEX1[3]	PIN_Y22	Seven Segment Digit 1[3]	d[1]
HEX1[4]	PIN_AA24	Seven Segment Digit 1[4]	e[1]
HEX1[5]	PIN_AA23	Seven Segment Digit 1[5]	f[1]
HEX1[6]	PIN_AB24	Seven Segment Digit 1[6]	g[1]
HEX2[0]	PIN_AB23	Seven Segment Digit 2[0]	a[2]
HEX2[1]	PIN_V22	Seven Segment Digit 2[1]	b[2]
HEX2[2]	PIN_AC25	Seven Segment Digit 2[2]	c[2]
HEX2[3]	PIN_AC26	Seven Segment Digit 2[3]	d[2]
HEX2[4]	PIN_AB26	Seven Segment Digit 2[4]	e[2]
HEX2[5]	PIN_AB25	Seven Segment Digit 2[5]	f[2]
HEX2[6]	PIN_Y24	Seven Segment Digit 2[6]	g[2]

5.3.1 Figuras

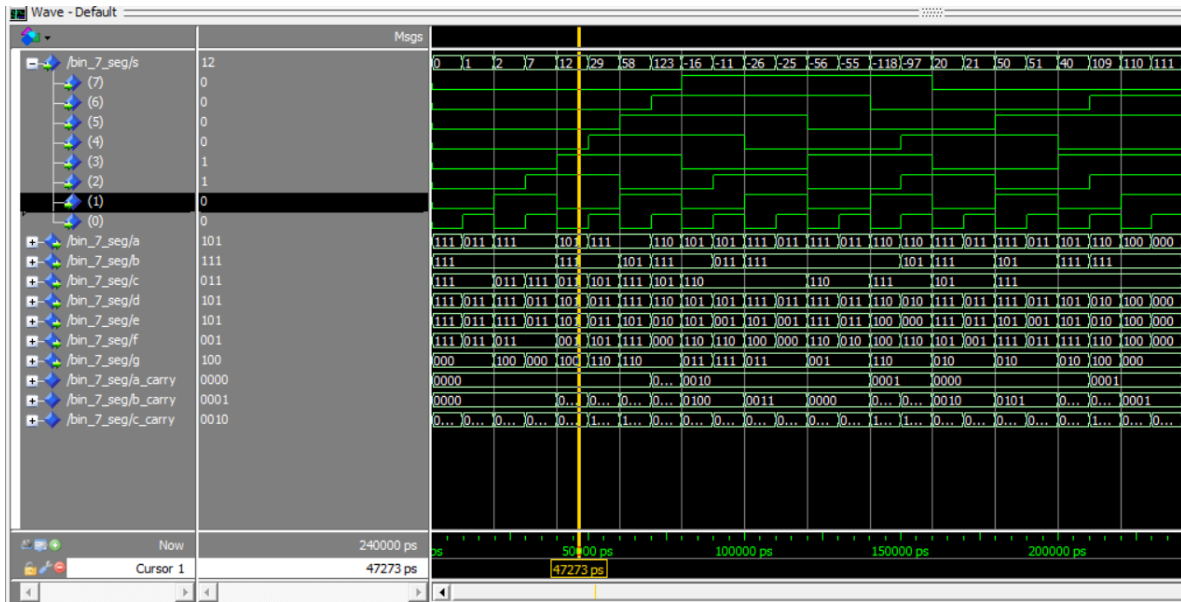


Figura 5.1: Bloco BIN-7-SEG.

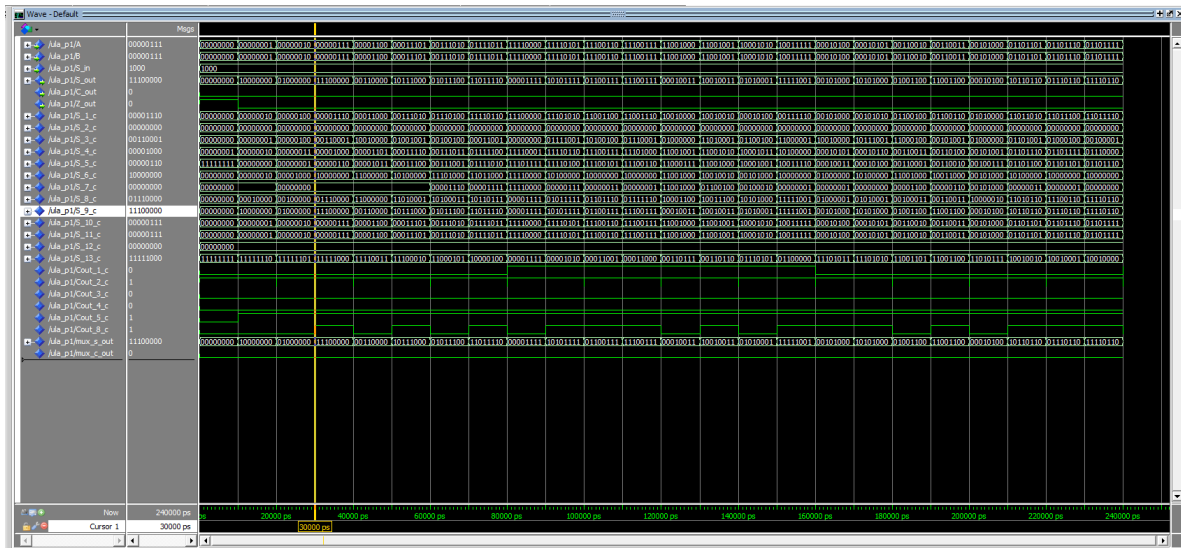


Figura 5.2: Bloco funções.

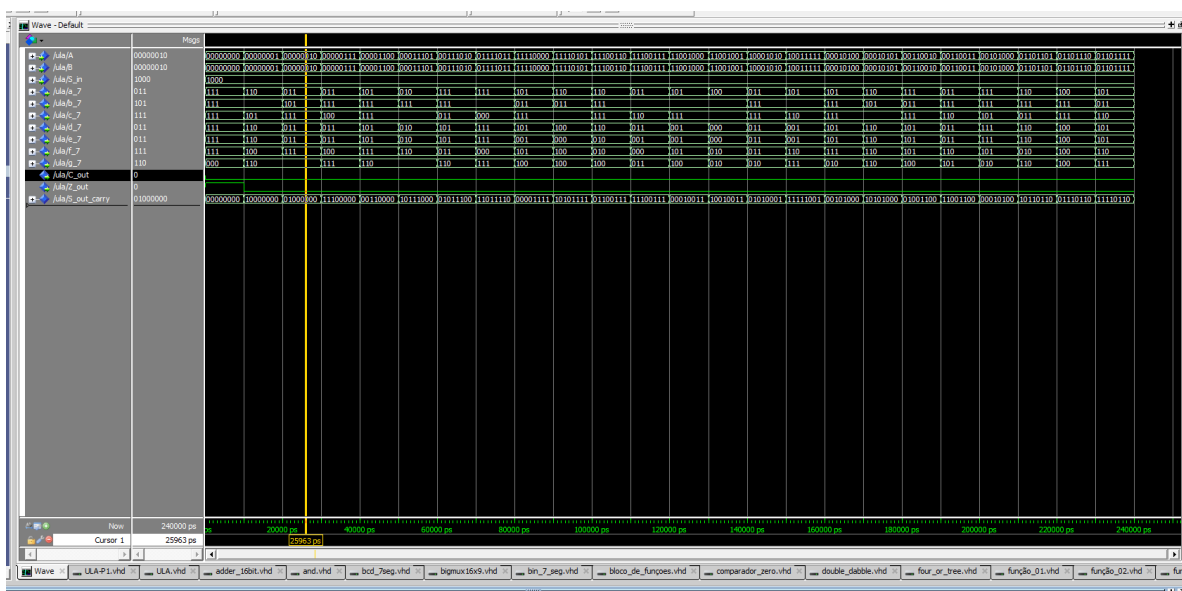


Figura 5.3: ULA completa.

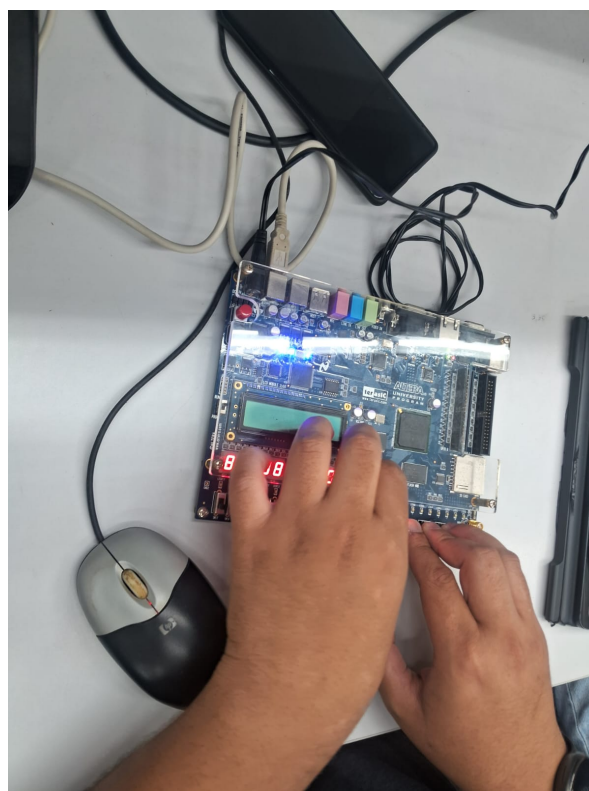


Figura 5.4: JPGA com a ULA.

6 Conclusão

A implementação da ULA em VHDL apresentou resultados satisfatórios, confirmando o funcionamento correto dos módulos aritméticos, lógicos e de controle desenvolvidos. Todas as funções especificadas foram sintetizadas e validadas em simulação. O sistema foi estruturado de forma modular, contemplando tanto as operações básicas quanto os blocos auxiliares de conversão e seleção, o que permitiu representar de maneira simplificada o funcionamento de uma ULA real.

Para garantir maior eficiência, foram aplicadas técnicas como o algoritmo Double-Dabble e a otimização por mapas de Karnaugh, resultando em circuitos de menor complexidade. Na etapa de testes em hardware, foi necessário realizar ajustes devido às características da placa utilizada que divergia da lógica inicialmente implementada. A inversão dos sinais corrigiu esses problemas e assegurou o funcionamento esperado.

Conclui-se que a ULA implementada atende plenamente às especificações estabelecidas, apresentando desempenho consistente tanto em simulação quanto na execução prática em FPGA. O projeto confirmou a viabilidade da arquitetura descrita, a compatibilidade com os recursos da plataforma utilizada e a correção da lógica desenvolvida, assegurando a fidelidade entre o modelo teórico e a aplicação em hardware.

Referências Bibliográficas

Falconer 2004 FALCONER, C. B. *An Explanation of the Double-Dabble Bin-BCD Conversion Algorithm*. 2004. Acesso em: 28 ago. 2025. Disponível em: <<https://web.archive.org/web/20090325002523/http://cbfalconer.home.att.net/download/dubldabl.txt>>.

Vahid 2008 VAHID, F. *Sistemas digitais: projeto, otimização e HDLs*. Porto Alegre: Artmed, 2008. 560p; 25 cm. ISBN 978-85-7780-190-9.

A Relato semanal - Semana 1

A.1 Equipe

Função no grupo	Nome completo do aluno	Responsável por
Líder:	Karen Helloisa Araújo Silva	Funções: 1, 4, 11, 13. Relatório.
Redator:	Eduardo Lira da Silva Filho	Função: 2. Relatório.
Debatedor	Rinaldo Tavares da Silva Filho	Funções: 3, 5, 8, 9, 10, 12, 14, 15. Relatório.

A.2 Defina o problema

Projeto da implementação de uma Unidade Logica Aritmética (ULA).

A.3 Pontos-chaves

- Projeto de todas funções aritméticas e lógicas;
- Circuitos de conversão Binário/BCD e BCD/Display 7 Segmentos.

A.4 Questões de pesquisa

- Algoritmo Double-Dabble.
- Conversor BDC para 7 segmentos.

A.5 Planejamento da pesquisa

Utilização de referências bibliográficas e sites.

B Relato semanal - Semana 2

B.1 Equipe

Função grupo	no	Nome completo do aluno	Responsável por
Líder		Eduardo Lira da Silva Filho	Funções: 10 Funções: 11 Funções: 12 Funções: 13 Funções: 14 Funções: 15 Relatório
Redator		Rinaldo Tavares da Silva Filho	Função: 3 Função: 8 Função: 9 Módulo ULA Módulo Conversores Relatório
Debatedor		Karen Helloisa Araújo Silva	Função: 1 Função: 2 Função: 4 Função: 5 Módulo Funções MUX 16X9 Relatório

C Anexos

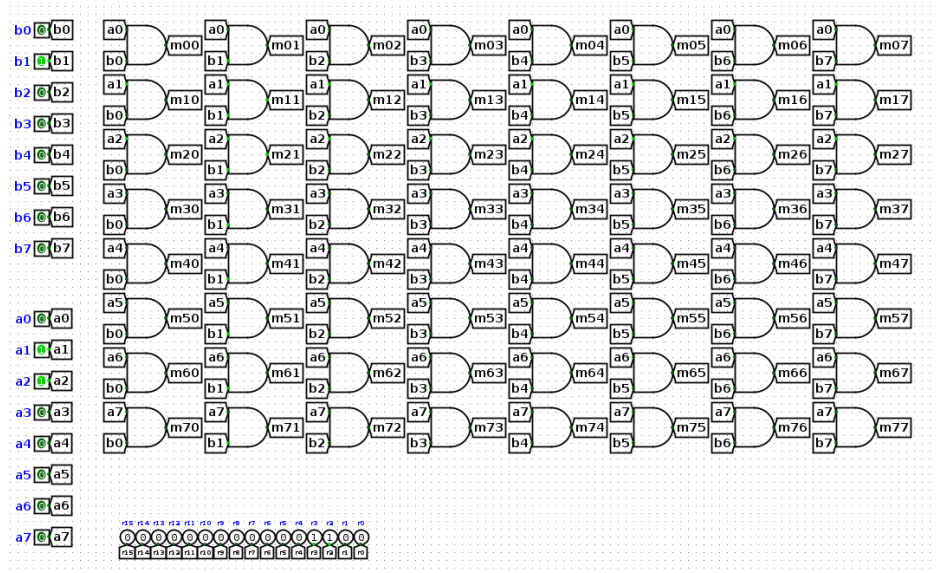


Figura C.1: Função 3º matriz do multiplicador.

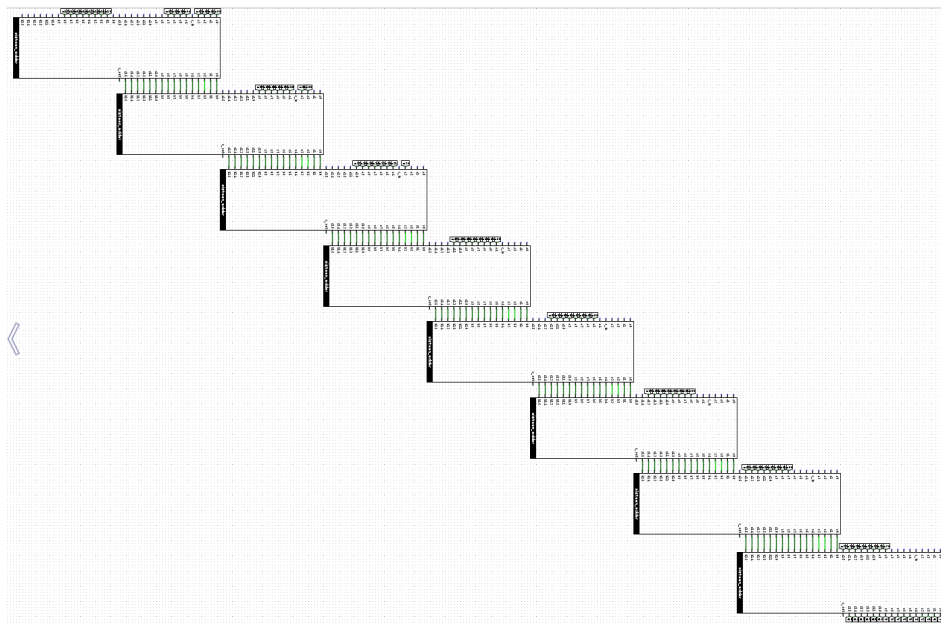


Figura C.2: Função 3º cadeia de somadores do multiplicador.

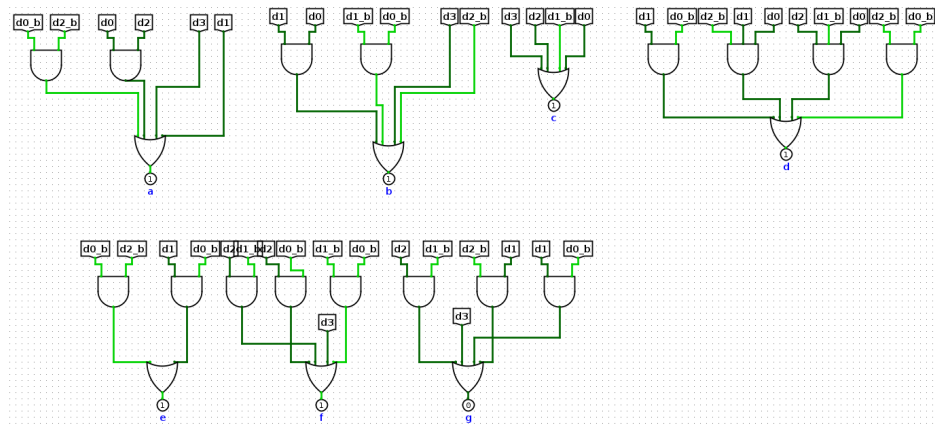


Figura C.3: Circuito do conversor BCD para 7 segmentos.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity comparador_zero is
5      port(a: in std_logic_vector(7 downto 0);
6            z: out std_logic);
7  end comparador_zero;
8
9  architecture ckt of comparador_zero is
10
11  begin
12
13      z <= not(a(0) or a(1) or a(2) or a(3) or a(4) or a(5) or a(6) or a(7));
14
15  end ckt;
16

```

Figura C.4: Comparador do Zero VHDL.

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity four_or_three is
5      port(a      : in std_logic_vector(3 downto 0);
6            s      : out std_logic_vector(3 downto 0));
7  end four_or_three;
8
9  architecture ckt of four_or_three is
10  begin
11      s(0) <= ((not a(0)) and a(3)) or ((not a(0)) and a(1) and a(2)) or (a(0) and (not a(2)) and (not a(3)));
12
13      s(1) <= ((not a(0)) and (not a(1)) and a(3)) or (a(1) and (not a(2)) and (not a(3))) or (a(0) and a(1));
14
15      s(2) <= ((not a(0)) and (not a(1)) and a(2)) or (a(1) and (not a(2)) and a(3)) or (a(0) and (not a(2)) and a(3));
16
17      s(3) <= ((not a(0)) and (not a(1)) and a(3)) or ((not a(2)) and a(3)) or (a(1) and a(2) and (not a(3))) or (a(0) and a(2) and (not a(3)));
18
19  end ckt;

```

Figura C.5: Four or three Otimizado VHDL.